第 40 章：Metaclasses and Inheritance（元类与继承）

> 原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：全书最后一章技术内容——在第 39 章装饰器（decorators）之后，把「在 class 语句收尾时自动插入代码」的模型推进到元类（metaclasses），并正式收官 Python 完整继承算法（含描述符偏误、赋值附录、super 补充、内置分岔）。本章同时是装饰器与元类的对照终章：两者在「管理类」上高度重叠，但元类还提供仅对类可见的次级继承树与元类方法。学完本章，你将具备「用代码管理类本身」的能力，并彻底看清属性查找在实例 / 类 / 元类三层上的全貌。

40.1 开篇引言（Opening）

英文原文

In Chapter 39, we explored decorators and studied examples of their use. In this final technical chapter of the book, we're going to continue our tool-builders focus with an in-depth review of another advanced topic: metaclasses, a protocol for managing class objects

instead of their instances, introduced briefly in Chapter 32. On a base level, metaclasses extend the code-insertion model of decorators. As we learned in the prior chapter, decorators allow us to augment functions and classes by intercepting their creation. Metaclasses similarly allow us to intercept and augment class creation—they provide a hook for inserting extra logic to be run at the conclusion of a class statement, albeit in different ways than decorators. Metaclasses can also provide behavior for classes with methods located in a separate inheritance tree skipped for normal, nonclass instances. While this allows metaclasses to process their instance classes after creation, it also compounds class semantics and convolutes inheritance—whose full definition can finally be fleshed out here. Like all the subjects covered in this part of the book, this is an advanced topic that can be studied on an as-needed basis. Metaclasses are not generally in scope for most application programmers but may be of interest to others seeking to write flexible tools. Whatever category you fall into, though, metaclasses can teach you more about Python's classes and are a prerequisite to both code that employs them and the complete inheritance story in Python. As the last technical chapter of this book, this also begins to wrap up some threads concerning Python itself that we have met often along the way and will finalize in the conclusion that follows. Where you go after this book is up t

o you, but in an open source project, it's important t o keep the big picture in mind while hacking the sma ll details.

中文翻译

在第 39 章中，我们探索了装饰器（decorators）并研究了它们的用法示例。在本书这最后一章技术内容里，我们将继续以「工具构建者（tool builders）」为焦点，深入审视另一个高级主题：元类（metaclasses）——一种用于管理类对象（class objects）而非其实例的协议，曾在第 32 章简要介绍过。在基础层面上，元类扩展了装饰器的「代码插入」模型。正如上一章所学，装饰器通过拦截函数和类的创建来增强它们。元类同样允许我们拦截并增强类的创建——它们提供一个钩子，在 class 语句收尾时插入额外逻辑，尽管方式与装饰器不同。元类还可以通过位于一棵独立继承树中的方法为类提供行为；这棵树对普通的、非类实例（nonclass instances）是跳过的。这让元类能在创建后处理其「实例类」，但同时也使类语义更复杂、继承更曲折——而继承的完整定义终于可以在本章定稿。与本书这一部分的所有主题一样，这是一个可按需学习的高级话题。元类通常不在大多数应用程序员的日常范围内，但对想编写灵活工具的人可能很有吸引力。无论你属于哪一类，元类都能让你更深入理解 Python 的类，并且是阅读使用元类的代码、以及理解 Python 完整继承故事的前提。作为本书最后一章技术内容，本章也开始收束我们一路反复遇到、将在随后结论章中最终定论的若干关于 Python 本身的线索。读完本书后去向何方取决于你；但在开源项目中，在钻研细节的同时保持大局观很重要。

深度理解

·核心概念：元类是「管理类对象」的协议；装饰器管理函数/类创建，元类在 class 语句结束时拦截类创建，并可提供仅对类可见的次级继承树行为。

·底层实现：class 语句收尾时调用 type（或自定义元类）的 call → new / init；元类方法落在「type 树」上，非类实例查找时不走这棵树。

·设计原因：把「增强一整批类」的横切逻辑从每个客户端类中抽离到一处声明（metaclass=），与装饰器的名字重绑定形成对照。

·解决什么实际问题：追踪、持久化、接口校验、批量给方法加装饰器、ORM/AOP 等工具层需求；更重要的是 demystify 类机制与完整继承。

·初学者误区：以为元类是日常业务必备。Lutz 明确说：多数应用程序员不必写元类；但要读懂框架与完整继承，必须理解它。

40.2 用不用元类（To Metaclass or Not to Metaclass）

英文原文

> Despite the advanced status awarded to metaclasses in the preceding opener, they have a variety of potential roles. For example, they can be used to enhance c

lasses with features like tracing, object persistence, exception logging, and more. They can also be used to construct portions of a class at runtime based on configuration files, apply function decorators to every method of a class generically, verify conformance to expected interfaces, and so on. In their more grandiose incarnations, metaclasses can even be used to implement alternative coding patterns such as aspect-oriented programming, object/relational mappers (ORMs) for databases, and more. Although there are often alternative ways to achieve such results—as you'll see, the roles of class decorators and metaclasses often intersect—metaclasses provide a formal model tailored to those tasks. We don't have space to explore all such applications firsthand in this chapter, of course, but you can find additional use cases on the web after studying the basics here. Probably the reason for studying metaclasses most relevant to this book is that this topic can help demystify Python's class mechanics in general. For instance, you'll find that they are an intrinsic part of the language's inheritance model formalized in full here. Although you may or may not code or reuse them in your work, a cursory understanding of metaclasses can impart a deeper understanding of Python at large.

中文翻译

> 尽管开篇给了元类「高级」的定位，它们仍有多种潜在角色。例如：为类增强追踪（tracing）、对象持久化、异常日志等功能；也可在运行时根据配置文件构造类的一部分、通用地把函数装饰器应用到类的每个方法、校验是否符合期望接口，等等。在更宏大的形态下，元类甚至可用于实现面向切面编程（AOP）、数据库的对象关系映射（ORM）等替代编码模式。虽然往往另有实现途径——你会看到，类装饰器与元类的角色经常相交——元类仍为这些任务提供了量身定制的正式模型。本章当然没有篇幅亲手探索全部应用，但在这里学完基础后，你可以在网上找到更多用例。对本书而言，学习元类最相关的理由或许是：它能帮助揭开 Python 类机制的神秘面纱。例如，你会发现元类是本章完整形式化的继承模型中固有的一部分。无论工作中是否编写或复用元类，粗浅理解它都能加深对 Python 整体的认识。

深度理解

·核心概念：元类的「用不用」取决于你是工具作者还是应用作者；用途从追踪/日志到 ORM/AOP 都有。

·底层实现：这些角色的共同点是「在类创建完成瞬间」统一插手——改命名空间、包方法、注册类。

·设计原因：提供比 helper 函数更显式、更难遗漏的 API 契约点（metaclass=）。

·解决什么实际问题：批量增强、运行时配置驱动的类装配、框架级横切。

·初学者误区：把「网上有人用元类做 X」当成自己业务代码也必须用元类；多数 X 也可用类装饰器完成。

40.3 「辅助函数」的弊端（The Downside of “Helper” Functions）

英文原文

Before we get to metaclass code, let's get a better handle on its rationale. Like the decorators of the prior chapter, metaclasses are optional in principle. We can usually achieve the same effect by passing class objects through functions—known interchangeably as helper or manager functions—much as we can achieve the goals of decorators by passing functions and classes through manager code. Just like decorators, though, metaclasses: Provide a more uniform and explicit structure; Help ensure that application programmers won't forget to augment their classes according to an API's requirements; Avoid code redundancy and its associated maintenance costs by factoring class customization logic into a single location. To illustrate, suppose we want to automatically insert a method into a set of classes. Of course, we could do this with simple inheritance if the subject method is known when we code the classes. In that case, we can simply code the method in a superclass and have all the classes in question inherit from it. Sometimes, though, it's impossible to predict such augmentation when classes are coded. Consider the case where classes are augmented in response to choices made in a user interface at

runtime or loaded from an editable configuration file. Although we could code every class in our imaginary set to manually check these, too, it's a lot to ask of cli ents. We can add methods to a class after the class st atement like this because, as we've learned, a class-le vel method is just a plain function that is associated with a class and has a first argument to receive a self instance when called through one. Although this wor ks, it might become untenable for larger method sets and puts all the burden of augmentation on each clie nt class (and assumes they'll remember to do this at all). It would be better from a maintenance perspecti ve to isolate the decision logic in a single place. We might encapsulate some of this extra work by routing classes through a helper function—a function that w ould extend the class as required and handle all the work of runtime testing and configuration. This code runs the class through a helper function immediately after it is created. Although functions like this one ca n achieve our goal here, they still put a burden on cla ss coders, who must understand the requirements an d adhere to them in their code. It would be even bett er if there was a simple way to enforce the augmenta tion in the subject classes, so that they don't need to deal with the augmentation so explicitly and would b e less likely to forget to use it altogether. In other wo rds, we'd like to be able to insert some code to run a utomatically at the end of a class statement to augm

ent the class. This is exactly what metaclasses do—by declaring a metaclass, we tell Python to route the cre ation of the class object to another class we provide. Because Python invokes the metaclass automatically at the end of the class statement when the new class is created, it can augment, register, wrap, or otherwis e manage the class as needed. Moreover, the only re quirement for the client classes is that they declare t heir metaclass; every class that does so will automati cally acquire whatever augmentation the metaclass p rovides, both now and in the future if the metaclass c hanges. Metaclasses can also augment their class inst ances with inherited methods, which are akin to nor mal class methods but not inherited by the instances of their class instances—an inheritance extension and tongue twister whose true nature requires more info ahead (and quite possibly, sedation). Through both c hanges and methods, though, metaclasses can custo mize class behavior broadly. Of course, this is the sta ndard rationale, which you'll need to judge for yours elf—in truth, clients might forget to list a metaclass j ust as easily as they could forget to call a helper func tion! Still, the explicit nature of metaclasses may mak e this less likely. Although it may be difficult to glean from this small and hypothetical example, metaclasse s generally handle such tasks better than more manu al approaches.

中文翻译

在进入元类代码之前，先更好地把握其动机。与上一章的装饰器一样，元类原则上是可选的。我们通常可以把类对象传给函数——可互换地称为辅助函数（helper）或管理函数（manager）——达到同样效果，正如装饰器的目标也可通过把函数和类传给管理代码实现。不过，就像装饰器一样，元类能：提供更统一、更显式的结构；帮助确保应用程序员不会忘记按 API 要求增强其类；通过把类定制逻辑提炼到单一位置，避免代码冗余及其维护成本。为说明，假设我们想自动向一组类中插入一个方法。当然，如果编写类时就已知该方法，可用简单继承：把方法写在超类中，让相关类都继承它。但有时在编码时无法预测这种增强——例如类根据运行时 UI 选择或可编辑配置文件来增强。虽然也可让每个客户端类手动检查，但对客户端要求过高。我们可以在 class 语句之后给类添加方法，因为类级方法本质上只是关联到类上的普通函数，通过实例调用时第一个参数接收 self。这能工作，但对更大的方法集可能难以为继，并把增强负担全压在每个客户端类上（还假设它们记得做）。从维护角度看，最好把决策逻辑隔离到一处。我们可以把类路由通过辅助函数——按需扩展类，并处理全部运行时测试与配置工作。这段代码在类创建后立即把它送进辅助函数。虽然能达成目标，仍给类编写者负担：必须理解并遵守要求。若有简单办法在目标类上强制执行增强，使它们不必如此显式处理、更不易完全忘记使用，会更好。换句话说，我们希望能在 class 语句末尾自动插入代码来增强类。这正是元类所做的——通过声明元类，我们告诉 Python 把类对象的创建路由到我们提供的另一个类。因为 Python 在 class 语句结

> 束、新类创建时自动调用元类，它可以按需增强、注册、包装或以其他方式管理类。客户端类的唯一要求是声明其元类；凡声明者都会自动获得元类提供的增强——现在如此，将来元类变更时亦然。元类还可通过继承方法增强其「类实例」；这些方法类似普通类方法，但不会被「类实例的实例」继承——这是一种继承扩展，也是绕口令，其真实本质需要后文更多信息（或许还需要镇静剂）。不过通过修改与方法，元类可以广泛定制类行为。当然，这是标准说辞，需你自行判断——说实话，客户端忘记写 metaclass= 与忘记调用辅助函数一样容易！不过元类的显式性可能使这更不可能。虽从这个小而假想的例子难以完全看出，元类通常比更手工的方法更好地处理此类任务。

深度理解

·核心概念：从「静态继承 → 客户端手工挂方法 → helper 函数 → 元类自动钩子」四级演进，动机是强制与集中。

·底层实现：class Client(metaclass=Extras) 在 class 收尾时等价于 Client = Extras(name, bases, dict)，在 init/new 里可 Class.extra = extra。

·设计原因：API 作者希望「声明一次、处处生效」；helper 依赖自觉调用，元类把调用绑在语法上。

·解决什么实际问题：运行时配置驱动的方法注入、减少客户端样板与遗忘。

·初学者误区：以为元类比 helper「绝对不会忘」——Lutz 诚实指出仍可能忘写 metaclass=；只是更显眼。

代码分析

```
# 1)
class Extras:
    def extra(self, args): ...

class Client1(Extras): ...
X = Client1()
X.extra()

# 2)
def extra(self, arg): ...
class Client1: ...
if required():
    Client1.extra = extra

# 3)
def extras(Class):
    if required():
        Class.extra = extra

class Client1: ...
extras(Client1)

# 4) class
class Extras(type):
    def __init__(Class, classname, superclasses, attributedict):
        if required():
            Class.extra = extra

class Client1(metaclass=Extras): ...
X = Client1()
X.extra()
```

解释器路径：定义 Client1 时，解释器收集类体命名空间 → 调用 Extras('Client1', bases, ns) → type.call 分派 n

ew/init → 在 init 中按条件挂上 extra。之后 X.extra() 走的是普通实例继承（方法已在类字典里），不是元类方法树。设计思想：把「是否增强」的决策从每个客户端挪到元类一处。

40.4 元类 vs 类装饰器：第一回合（Metaclasses Versus Class Decorators: Round 1）

英文原文

> Having said that, it's also important to note that the class decorators described in the preceding chapter sometimes overlap with metaclasses—in terms of both utility and benefit. Like metaclasses, class decorators can be used to manage both classes and their later instances, and their syntax makes their usage similarly explicit and arguably more obvious than helper-function calls. For example, suppose we recoded the last section's helper function to return the augmented class instead of simply modifying it in place. This would allow a greater degree of flexibility because the manager would be free to return any type of object that implements the class's expected interface. If you think this is starting to look reminiscent of class decorators, you're right. In the prior chapter, we emphasized class decorators' role in augmenting instance creation calls. Because they work by automatically rebinding a class name to the result of a function, though, there's

no reason that we can't use them to augment the cla ss by changing it before any instances are ever create d. That is, class decorators can apply extra logic to cl asses, not just instances, at class creation time. Decor ators essentially automate the prior example's manua l name rebinding here. Just as for metaclasses, becau se this decorator returns the original class, instances are made from that class, not from a wrapper object. In fact, instance creation is not intercepted at all in th is example. In this specific case—adding methods to a class when it's created—the choice between metacl asses and decorators is arbitrary. Decorators can be u sed to manage both instances and classes and inters ect most strongly with metaclasses in the second of t hese roles, but this discrimination is not absolute. In f act, the roles of each are suggested in part by their mechanics. As we'll detail ahead, decorators technical ly correspond to metaclass calls used to make and ini tialize new classes. Metaclasses, though, have additio nal customization hooks beyond class creation. Their methods inherited by classes, for example, have no di rect counterpart in class decorators sans extra code. This can make metaclasses more complex but also be tter suited for augmenting classes in some contexts. Conversely, because metaclasses are designed to ma nage classes, applying them to managing instances al one is less optimal. Because they are also responsible for making the class itself, metaclasses incur this as a

n extra step in instance-management roles and are perhaps less appropriate than decorators. We'll explore some of these differences in working code later in this chapter. To better understand how metaclasses do their work, though, we first need to get a clearer picture of their underlying model.

中文翻译

话虽如此，也必须指出：上一章的类装饰器在效用与收益上有时与元类重叠。像元类一样，类装饰器既可管理类也可管理其后的实例，其语法同样显式，且arguably 比 helper 调用更一目了然。例如，假设把上节 helper 改成返回增强后的类，而不是原地修改。这样更灵活，因为管理器可自由返回任何实现了类期望接口的对象。如果你觉得这开始像类装饰器，你是对的。上章我们强调类装饰器增强实例创建调用；但因其通过自动把类名重绑定到函数结果工作，没有理由不能在创建任何实例之前就改类来增强它。也就是说，类装饰器可在类创建时对类（不只是实例）施加额外逻辑。装饰器本质上自动化了前例的手动名字重绑定。与元类一样，因装饰器返回原类，实例由该类创建，而非包装对象。本例甚至完全不拦截实例创建。在这个具体情形——创建时给类加方法——元类与装饰器的选择是任意的。装饰器可管理实例与类，与元类在「管理类」角色上重叠最强，但划分并非绝对；各角色部分由其机制暗示。后文将详述：装饰器在技术上对应「创建并初始化新类」的元类调用。但元类在类创建之外还有额外定制钩子。例如被类继承的元类方法，在类装饰器中没有直接对等物（除非额外写代码）。这使元类更复杂，但在某些上下文中更适于增强类。反过来，因元

类为管理类而设计，仅用它管理实例并不最优。它们还负责制造类本身，在「只管实例」角色里多出一步，或许不如装饰器合适。本章稍后会用可运行代码探索部分差异。但为更好理解元类如何工作，我们先需要更清晰的底层模型。

深度理解

·核心概念：Round 1 结论——「给类加方法」时装饰器 ≈ 元类；差异在机制与额外钩子（元类方法、创建协议）。

·底层实现：@extras class C ≡ C = extras(C)；元类是 C = Meta(name, bases, dict)。装饰器在类已建成后重绑定；元类可在 new 建成前改 dict。

·设计原因：装饰器对应「创建后处理」；元类横跨「创建中 + 创建后 + 类级行为」。

·解决什么实际问题：选型指南——只管实例优先装饰器；要管类创建协议/元类方法用元类。

·初学者误区：二选一教条。Lutz 的立场是角色重叠但力学不同，按钩子需求选。

代码分析

```
def extra(self, arg): ...

def extras(Class):                  # helper →
    if required():
        Class.extra = extra
    return Class

class Client1: ...
Client1 = extras(Client1)           #
```

```
@extras                          #
class Client2: ...
# Client2 = extras(Client2)

X = Client1()
X.extra()                        #
```

设计思想：当装饰器返回原类时，类型身份保留（isinstance 仍认 Client1）；这与「返回代理包装实例」的类装饰器是两条路。加方法场景走「改类并返回原类」，故与元类几乎同构。

40.5 元类模型（The Metaclass Model）

英文原文

> To understand metaclasses, you first need to understand a bit more about both Python's object model and what happens at the end of a class statement. As you'll learn here, the two are intimately related.

中文翻译

> 要理解元类，你首先需要更多了解 Python 的对象模型，以及 class 语句末尾发生了什么。你将在此学到：二者紧密相关。

深度理解

·核心概念：对象模型（谁是谁的实例）与 class 收尾协议（谁被调用来造类）是元类的两根支柱。

·底层实现：后续五小节分别展开 type 实例关系、子类化 type、type(...) 调用、metaclass= 选择、new/init 协议。

·设计原因：先建模型再写代码，避免把元类当成「魔法 @ 的亲戚」死记。

·解决什么实际问题：为「Coding Metaclasses」提供语义坐标系。

·初学者误区：跳过模型直接抄 class M(type) 模板——一改协议就迷路。

40.6 类是 type 的实例（Classes Are Instances of type）

英文原文

> The first of these prerequisites was covered previously by "The Python Object Model", which in turn assumes knowledge of the MRO (method resolution order) covered in Chapter 31. You should review that content now if needed, especially if you've jumped into this chapter at random. We're not going to repeat its coverage in full here, but some of its conclusions are crucial to understanding metaclasses. Namely: Instances are created from classes. Classes are instances of a metaclass. The type built-in is the topmost metaclass. Metaclasses customize type with normal class statements. In short, classes are types, types are classes, and

metaclasses customize types. Per Chapter 32, the rela tionship between metaclasses and classes is subtly di fferent from that between classes and their nonclass i nstances. The latter do not generate more instances, and while attribute inheritance uses the same core m echanisms and MRO everywhere, classes search a me taclass tree that nonclass instances do not. We'll stud y the nuts and bolts of inheritance ahead, but it's eas y to see this model's fundamentals in code. Built-in o bjects like lists are actually nonclass instances made f rom a class, which itself is made from the built-in typ e. Apart from the literal syntax of built-ins, this works the same way for user-defined classes, which are reall y just user-defined types—their nonclass instances ar e made from the class, which itself is made from the built-in type. Although their behavior varies in ways we explored in Chapter 32, both classes and nonclass instances are "instances" in some sense. In fact, the cl ass attribute in both tells us what they were made fro m. Because classes are created from the root type cla ss by default, most programmers don't need to think about this model. However, it's key to understanding the way that metaclasses work—as the next section e xplains.

中文翻译

这些前提中的第一项此前已在「Python 对象模型」中讲过，而那又假定你掌握第 31 章的 MRO（方法解析顺序）。

若需要请先复习，尤其是若你随机跳进本章。这里不完整重复，但若干结论对理解元类至关重要：实例由类创建；类是元类的实例；内建 type 是最顶层元类；元类用普通 class 语句定制 type。简言之：类是类型，类型是类，元类定制类型。按第 32 章，元类与类的关系，与类与其非类实例的关系，微妙不同。后者不会再生成更多实例；虽属性继承处处使用相同核心机制与 MRO，类会搜索一棵非类实例不搜索的元类树。后文将研究继承细节，但用代码很容易看到模型基础。像列表这样的内建对象，其实是由某个类造出的非类实例，而该类本身由内建 type 造出。除内建的字面量语法外，用户定义类同样如此——它们其实就是用户定义类型：其非类实例由类造出，类本身由内建 type 造出。虽行为在第 32 章所述方面有差异，类与非类实例在某种意义上都是「实例」。事实上，二者的 class 属性都告诉我们它们由什么造出。因为类默认由根 type 类创建，多数程序员不必思考此模型。但它是理解元类工作方式的关键——下一节解释。

深度理解

·核心概念：type(obj) / obj.class 揭示「实例 ← 类 ← type」链条；type 的类型是 type 自身（层级顶点）。

·底层实现：list 是类；[] 是 list 的实例；list 是 type 的实例。用户类 Hack 同理。

·设计原因：统一「一切皆对象」：类也是对象，故可被「造类的类」管理。

·解决什么实际问题：解释为何能写 class Meta(type) 并介入类创建。

·初学者误区：把「类是 type 的实例」与「类继承 object」混为一谈——那是两条不同关系（instance-of vs subclass-of）。

代码分析

```
>>> type([]), type(type([]))
(<class 'list'>, <class 'type'>)   #  ← list  ← type

>>> type(list), type(type)
(<class 'type'>, <class 'type'>)   # type  type

>>> class Hack: pass
>>> I = Hack()
>>> type(I), type(type(I))
(<class '__main__.Hack'>, <class 'type'>)

>>> [].__class__, list.__class__
(<class 'list'>, <class 'type'>)
>>> I.__class__, Hack.__class__
(<class '__main__.Hack'>, <class 'type'>)
#  type(X)  X.__class__
```

设计思想：大多数程序员默认走 type 造类，感觉不到元类；一旦替换 type 的子类，整条「造类」管道就被接管。

40.7 元类是 type 的子类（Metaclasses Are Subclasses of type）

英文原文

Why would we care that classes are instances of a type class? It turns out that this is the hook that allows us to code metaclasses. Specifically, we can create classes from subclasses of type that customize it with normal object-oriented techniques and class syntax. And these type subclasses are known as metaclasses. In other words, to control the way classes are created and augment their behavior, all we need to do is specify that a user-defined class be created from a user-defined metaclass instead of the normal and default type class. Before we see how, it's important to bear in mind that this type instance relationship is not quite the same as normal inheritance. User-defined classes may also have superclasses from which they and their instances inherit attributes as usual. As we've seen, inheritance superclasses are listed in parentheses in the class statement, show up in a class's bases tuple, and are searched for attributes fetched from nonclass instances. However, the type from which a class is created and of which it is an instance is a different relationship. Inheritance searches instance and class namespace dictionaries, but classes may also acquire behavior from their type that is not exposed to the normal inheritance search. In fact, metaclasses define a separate, secondary inheritance tree available only to classes and used as a fallback when the normal superclass

> search fails. To lay the groundwork for understanding this distinction, the next section describes the procedure and syntax Python uses to implement this instance-of relationship.

中文翻译

> 我们为何在意「类是 type 类的实例」？原来这正是编写元类的钩子。具体而言，我们可以从定制了 type 的子类创建类，用普通面向对象技巧与 class 语法；这些 type 子类就称为元类。换句话说，要控制类的创建方式并增强其行为，只需指定用户定义类由用户定义元类创建，而非默认的 type。在看如何做之前，务必记住：这种 type 实例关系与普通继承并不完全相同。用户定义类也可有超类，它们及其实例照常从中继承属性。如我们所见，继承超类列在 class 语句括号中，出现在类的 bases 元组里，并在从非类实例获取属性时被搜索。然而，「类由何种 type 创建、是何种 type 的实例」是另一种关系。继承搜索实例与类的命名空间字典，但类还可从其 type 获得行为，且该行为不暴露给普通继承搜索。事实上，元类定义了一棵独立的次级继承树，仅对类可用，并在普通超类搜索失败时作为回退。为理解这一区分打下基础，下一节描述 Python 用来实现这种 instance-of 关系的过程与语法。

深度理解

·核心概念：instance-of（元类）≠ subclass-of（超类）；前者走 class，后者走 bases/mro。

·底层实现：次级树 = 类的 class.mro（通常含自定义 Meta、type、object）；仅类查找会 fallback 到此树。

·设计原因：让「管类的行为」与「管实例的行为」命名空间隔离，避免元类方法泄漏到普通实例。

·解决什么实际问题：解释后文「Sub.meth3() 可调而 X.meth3() 报错」。

·初学者误区：把 metaclass=M 写成以为 M 是超类——超类要写在 metaclass= 之前的位置参数里。

40.8 class 语句会调用 type（Class Statements Call a type）

英文原文

Subclassing the type class to customize it is really only half of the metaclass backstory. We still need to somehow route a class's creation to the metaclass instead of the default type. To comprehend the way that this is arranged, we also need to know how class statements do their business. We've already learned that when Python reaches a class statement, it runs its nested block of code to create the class's attributes—all the names assigned at the top level of the nested code block generate attributes in the resulting class object. These names are usually method functions created by nested defs, but they can also be arbitrary attributes assigned to create class data shared by all instances. Technically speaking, Python follows a standard protocol to make this happen: at the end of a class st

atement, and after running all its nested code in a na
mespace dictionary corresponding to the class's local
scope, Python calls the type object to create the new
class object like this: class = type(classname, supercla
sses, attributedict). The type object in turn defines a
call operator-overloading method that runs two othe
r methods when the type object is called: type.new(ty
peclass, classname, superclasses, attributedict) then t
ype.init(class, classname, superclasses, attributedict).
The new method creates and returns the new class o
bject, after which the init method initializes the newly
created object. As you'll see in a moment, these are t
he hooks that metaclass subclasses of type generally
use to perform class customizations at creation time.
In fact, you can call type this way yourself to create a
class dynamically—albeit here with a fabricated meth
od function and empty superclasses tuple (Python ad
ds the object superclass automatically to topmost cla
sses as we learned in Chapter 32, and the enclosing
module's name is again implied). The class produced
by a direct type call is exactly like that you'd get from
running a class statement. Because this type call is m
ade automatically at the end of the class statement, t
hough, it's an ideal hook for augmenting or otherwis
e processing a class. The trick lies in replacing the def
ault type with a custom subclass that will intercept th
is call. The next section shows how.

中文翻译

> 子类化 type 以定制它其实只是元类故事的一半。我们仍需把类的创建路由到元类而非默认 type。要理解如何安排，还需知道 class 语句如何办事。我们已经学过：Python 到达 class 语句时，运行其嵌套代码块以创建类的属性——嵌套块顶层赋值的所有名字都成为结果类对象的属性。这些名字通常是嵌套 def 产生的方法函数，也可以是任意赋值，以创建所有实例共享的类数据。技术上，Python 遵循标准协议：在 class 语句末尾，于对应类局部作用域的命名空间字典中运行完全部嵌套代码之后，调用 type 对象创建新类：class = type(classname, superclasses, attributedict)。type 对象又定义了 call 运算符重载方法，在被调用时运行另外两个方法：type.new(typeclass, classname, superclasses, attributedict)，然后 type.init(class, classname, superclasses, attributedict)。new 创建并返回新类对象，随后 init 初始化已创建对象。稍后可见，这些正是 type 的元类子类通常用来在创建时执行类定制的钩子。事实上，你可以自己这样调用 type 来动态创建类——此处用捏造的方法函数与空超类元组（Python 会按第 32 章所述自动为最顶层类加上 object 超类，外围模块名再次隐含）。直接 type 调用产生的类与运行 class 语句得到的完全一样。正因 class 末尾自动发生这次 type 调用，它是增强或以其他方式处理类的理想钩子。诀窍在于用自定义子类替换默认 type 以拦截该调用。下一节展示如何做。

深度理解

·核心概念：class 语法糖 = 执行类体 + type(name, bas

es, dict)。

·底层实现：type.call → new（造）→ init（初始化）；元类重定义后两者即可介入。

·设计原因：把「声明式 class」与「命令式造类型」统一到同一协议，动态类与静态类同构。

·解决什么实际问题：运行时按配置拼类、ORM 模型类生成等。

·初学者误区：以为只有 class 关键字能造类——type(...) 完全等价。

代码分析

```
class Super: ...
class Hack(Super):
    data = 1
    def meth(self, arg):
        return self.data + arg

#  __module__
Hack = type('Hack', (Super,), {'data': 1, 'meth': meth, '__module_…

#
>>> c = type('Hack', (), {'data': 1, 'meth': (lambda x, y: x.data …
>>> i = c()
>>> i.data, i.meth(2)
(1, 3)
>>> c.__bases__
(<class 'object'>,)
>>> i.__class__.__mro__
(<class '__main__.Hack'>, <class 'object'>)
```

解释器：类体跑完得到 classdict，再一次函数调用风格的 type(...) 把三要素粘成类对象。设计思想：元类只要成为

「被调用的那个 callable」，就能接管整段协议。

40.9 class 语句可选择 type（Class Statements Can Choose a type）

英文原文

As we've just seen, classes are created by the type class by default. To tell Python to create a class with a custom metaclass instead, you simply need to declare a metaclass to intercept the normal instance creation call for a user-defined class. To do so, list the desired metaclass as a keyword argument in the class header: class Hack(metaclass=Meta): # Use Meta instead of type default. If no such declaration is present, the metaclass to be called defaults to the type built-in, per the prior section. When used, though, this declaration overrides the type default and routes the class creation call at the close of the class statement to Meta instead: class = Meta(classname, superclasses, attributedict). Importantly again, the metaclass keyword specifies an instance-of relationship, which implies inheritance only through the secondary metaclass tree we'll formalize ahead. Normal inheritance superclasses can be listed in the header as well and take precedence by residing in the primary class tree. In the following, for example, the new class Hack inherits from superclass Super normally but is also an instance of—and is c

reated by—metaclass Meta: class Hack(Super, metaclass=Meta): # Normal supers OK: listed first. In this form, superclasses must be listed before the metaclass; in effect, the ordering rules used for keyword arguments in function calls apply here too. This order also has implications for inheritance, which we'll formalize soon, but first, we need to learn how to code the metaclasses that tap into calls triggered by this special class syntax.

中文翻译

正如刚看到的，类默认由 type 类创建。若要让 Python 用自定义元类创建类，只需声明元类以拦截用户定义类的正常「实例创建」调用。做法是在 class 头中把所需元类列为关键字参数：class Hack(metaclass=Meta)：（用 Meta 代替默认 type）。若无此声明，被调用的元类默认是内建 type。一旦使用，该声明覆盖 type 默认，把 class 语句收尾处的类创建调用路由到 Meta：class = Meta(classname, superclasses, attributedict)。再次强调：metaclass 关键字指定的是 instance-of 关系，仅通过后文将形式化的次级元类树蕴含继承。普通继承超类也可列在头中，并因位于主类树而优先。例如：新类 Hack 正常继承超类 Super，同时也是元类 Meta 的实例并由其创建：class Hack(Super, metaclass=Meta)：（普通超类可以，且列在前面）。这种形式下，超类必须列在元类之前；实际上适用函数调用中关键字参数的排序规则。此顺序对继承也有影响，我们很快形式化；但首先需要学习如何编写接入这种特殊 class 语法所触发调用的元类。

深度理解

·核心概念：metaclass=Meta 改的是「造类的工厂」，不是「继承谁」。

·底层实现：收尾调用从 type(...) 变为 Meta(...)；Meta 通常仍是 type 子类以复用 call 分派。

·设计原因：关键字参数位置避免与基类列表混淆；基类在前、元类关键字在后。

·解决什么实际问题：同一继承树可混用不同元类策略（经子类继承 metaclass= 声明，后文述）。

·初学者误区：class C(Meta) 把元类当超类——应写 class C(metaclass=Meta)。

代码分析

```
class Hack(metaclass=Meta):
    ...
# Hack = Meta('Hack', bases, classdict)

class Hack(Super, metaclass=Meta):  # metaclass
    ...
```

40.10 元类方法协议（Metaclass Method Protocol）

英文原文

> When a specific metaclass is declared per the prior sections' syntax, the call to create the class object run a

t the end of the class statement is modified to invoke the metaclass instead of the type default: class = Meta(classname, superclasses, attributedict). Assuming the metaclass is a subclass of type, though, the type class's inherited call method delegates creation and initialization of the new class object to the metaclass if the metaclass defines custom versions of the methods that handle these steps. In other words, the Meta call may wind up triggering these method calls in turn: class = Meta.new(Meta, classname, superclasses, attributedict) then Meta.init(class, classname, superclasses, attributedict). To demonstrate, here's the preceding class example again, augmented with a metaclass specification. At the end of this class statement, Python internally runs the following to create the class object—again, a call you could make manually, too, but automatically run by Python's class machinery: Hack = Meta('Hack', (Super,), {'data': 1,'meth': meth,'module': 'main'}). If the metaclass defines its own versions of new or init, they will be invoked during this call by the inherited type class's call method. The net effect is to automatically run methods the metaclass provides as part of the class-construction process. The next section shows how we might go about coding this final piece of the metaclass puzzle.

中文翻译

> 当按前几节语法声明了特定元类时，class 语句末尾创建类对象的调用改为调用元类而非默认 type：class = Meta(classname, superclasses, attributedict)。若元类是 type 的子类，则从 type 继承的 call 会在元类自定义了处理这些步骤的方法时，把新类对象的创建与初始化委托给元类。换言之，对 Meta 的调用最终可能依次触发：class = Meta.new(Meta, classname, superclasses, attributedict)，然后 Meta.init(class, classname, superclasses, attributedict)。演示：前例加上元类规格。class 语句末尾 Python 内部运行：Hack = Meta('Hack', (Super,), {'data': 1,'meth': meth,'module':'main'})——你也可手动调用，但由类机制自动运行。若元类定义了自己的 new 或 init，它们会在此次调用中由继承的 type.call 调用。净效果是：作为类构造过程的一部分，自动运行元类提供的方法。下一节展示如何编写元类拼图的最后一块。

深度理解

·核心概念：协议三元组——call（调度）/ new（创建）/ init（初始化）。

·底层实现：第一个参数对 new 是元类本身（meta），对 init 是已创建的类（Class）。

·设计原因：与普通实例的 type(obj)(...) → new/init 同构，降低心智迁移成本。

·解决什么实际问题：在「类存在之前」改 classdict（new）或「类已存在之后」改类（init）。

·初学者误区：在 new 里忘记 return type.new(...)——类会变成 None。

代码分析

```
class Hack(Super, metaclass=Meta):
    data = 1
    def meth(self, arg):
        return self.data + arg


#
# Hack = Meta('Hack', (Super,), {'data': 1, 'meth': meth, '__modul…
#  Meta  type.__call__
#   Meta.__new__(Meta, 'Hack', (Super,), classdict)
#   Meta.__init__(Hack, 'Hack', (Super,), classdict)
```

40.11 编写元类（Coding Metaclasses）

英文原文

> So far, we've seen how Python routes class creation c
> alls to a metaclass if one is specified and provided. H
> ow, though, do we actually code a metaclass that cus
> tomizes type? It turns out that you already know mos
> t of the story—metaclasses are coded with normal Py
> thon class statements and semantics. By definition, th
> ey are simply classes that inherit from type (normally,
> at least). Their only substantial distinctions are that P
> ython calls them automatically at the end of a class st
> atement and that they must generally adhere to the i
> nterface expected by the type superclass if they subcl
> ass it.

中文翻译

> 到目前为止，我们已看到：若指定并提供了元类，Python 如何把类创建调用路由到元类。那么，如何实际编写定制 type 的元类？事实证明你已知道故事的大部分——元类用普通 Python class 语句与语义编写。按定义，它们只是（通常）继承自 type 的类。实质区别仅在于：Python 在 class 语句末尾自动调用它们；若子类化 type，一般须遵守 type 超类期望的接口。

深度理解

·核心概念：元类 = 普通类 + 继承 type + 自动在 class 收尾被调用。

·底层实现：遵守 new(meta, name, bases, dict) / init(cls, name, bases, dict) 签名。

·设计原因：复用已有 OOP，不发明第二套语法。

·解决什么实际问题：降低元类学习曲线——「你会写类就会写元类骨架」。

·初学者误区：把元类想成全新语言特性；其实只是约定角色的 type 子类。

40.12 一个基本元类（A Basic Metaclass）

英文原文

Perhaps the simplest metaclass you can code is simply a subclass of type with a new method that creates t

he class object by running the default method in typ e. A metaclass new like this is run by the call method inherited from type by virtue of normal inheritance o verrides; this method typically performs whatever au gmentation is required and calls the type superclass's new method to create and return the new class objec t. This metaclass doesn't really do anything (we migh t as well let the default type create the class), but it d emonstrates the way a metaclass taps into the metacl ass hook to customize—because the metaclass is call ed at the end of a class statement, and because the t ype object's call dispatches to the new and init meth ods, code we provide in these methods can manage all the classes created from the metaclass. Here, class Hack inherits from Super and is an instance of MetaO ne, but X is an instance of and inherits from Hack. W hen run, notice how the metaclass is invoked at the e nd of the class statement and before we ever make a n instance of the new class—metaclasses process clas ses, and classes process nonclass instances. Presentat ion note: this chapter's examples often truncate hex addresses and omit some irrelevant built-in X names in namespace dictionaries, both for brevity and beca use built-in attributes tend to change over time. Run these examples on your own for full, if transient, fidel ity.

中文翻译

> 你能编写的最简单元类，或许只是带 new 的 type 子类，通过运行 type 的默认方法创建类对象。这样的元类 new 由从 type 继承的 call 因普通继承覆盖而运行；该方法通常执行所需增强，再调用 type 超类的 new 创建并返回新类对象。这个元类其实什么也不做（大可让默认 type 创建类），但演示了元类如何接入钩子来定制——因为元类在 class 语句末尾被调用，且 type 的 call 分派到 new 与 init，我们在这些方法中提供的代码可以管理所有由该元类创建的类。此处，类 Hack 继承 Super 且是 MetaOne 的实例，但 X 是 Hack 的实例并从 Hack 继承。运行时注意：元类在 class 语句末尾、我们尚未创建新类任何实例之前就被调用——元类处理类，类处理非类实例。排版说明：本章示例常截断十六进制地址，并省略命名空间字典中某些无关的内建 X 名，既为简洁，也因内建属性会随时间变化。请自行运行示例以获得完整（即便短暂）的保真输出。

深度理解

·核心概念：最小元类 = type.new 转发 + 可选打印/增强。

·底层实现：MetaOne.new 在「Making instance」之前打印——证明钩子在类创建时而非实例创建时。

·设计原因：先展示时机与参数（meta, classname, supers, classdict），再谈有用增强。

·解决什么实际问题：调试类创建顺序、确认 metaclass 是否生效。

·初学者误区：在实例方法里找元类打印——打印发生在 import/定义阶段。

代码分析

```
# Example 40-1. metaclass1.py
class MetaOne(type):
    def __new__(meta, classname, supers, classdict):
        #  type.__call__
        print('In MetaOne.new:', meta, classname, supers, classdic…
        return type.__new__(meta, classname, supers, classdict)

class Super:
    pass

print('Making class')
class Hack(Super, metaclass=MetaOne):  #  Super MetaOne
    data = 1                           #
    def meth(self, arg):               #
        return self.data + arg

print('Making instance')
X = Hack()
print('Attrs:', X.data, X.meth(2))

#
# Making class
# In MetaOne.new:
# ...<class '__main__.MetaOne'>
# ...Hack
# ...(<class '__main__.Super'>,)
# ...{'__module__': '__main__', 'data': 1, 'meth': <function ...>}
# Making instance
# Attrs: 1 3
```

解释器：定义 Hack 时立即进入 MetaOne.new，参数带上类名、基类元组、类体字典；返回后 Hack 名字绑定完

成，此后 X = Hack() 才走普通实例构造。设计思想：元类时间线永远早于非类实例。

40.13 定制构造与初始化（Customizing Construction and Initialization）

英文原文

> Metaclasses can also tap into the init protocol invoked by the type object's call. In general, new creates and returns the class object, and init initializes the already created class passed in as an argument. These methods work in non-type classes, too, but new is rare in such classes. Metaclasses can use either or both hooks to manage classes at creation time. In this case, the class initialization method is run after the class construction method, but both methods run at the end of the class statement and before any nonclass instances are made. Conversely, an init in Hack would run later at nonclass-instance creation time and would not be affected or run by the metaclass's init.

中文翻译

元类也可接入由 type 对象的 call 调用的 init 协议。一般而言，new 创建并返回类对象，init 初始化作为参数传入的已创建类。这些方法在非 type 类中也可用，但 new 在此类中少见。元类可用二者之一或同时在创建时管理类。本例中，类初始化方法在类构造方法之后运行，但两者都在 cl

ass 语句末尾、任何非类实例创建之前运行。相反，Hack 中的 init 会在更晚的非类实例创建时运行，不受元类 init 影响，也不会被其运行。

深度理解

·核心概念：new 造类对象；init 初始化已存在的类；二者都在实例出现前。

·底层实现：init 的第一个参数已是类对象，可 list(Class.dict.keys()) 观察。

·设计原因：需要改 classdict 再建造时用 new；类已存在只需挂属性/注册时用 init 更简单。

·解决什么实际问题：注册表、校验最终类形态、打印调试。

·初学者误区：混淆元类 init 与用户类 init——前者管类，后者管实例。

代码分析

```
# Example 40-2. metaclass2.py
class MetaTwo(type):
    def __new__(meta, classname, supers, classdict):
        print()
        print('In MetaTwo.new:', meta, classname, supers, classdic…
        return type.__new__(meta, classname, supers, classdict)
    def __init__(Class, classname, supers, classdict):
        print()
        print('In MetaTwo.init:', Class, classname, supers, classd…
        print('...init class object:', list(Class.__dict__.keys()))

class Super:
    pass
```

```
print('Making class')
class Hack(Super, metaclass=MetaTwo):
    data = 1
    def meth(self, arg):
        return self.data + arg

print('\nMaking instance')
X = Hack()
print('Attrs:', X.data, X.meth(2))

# Making class → MetaTwo.new → MetaTwo.init → Making instance →…
```

40.14 其他元类编写技巧（Other Metaclass Coding Techniques）

英文原文

> Although redefining the type superclass's new and init methods is the most common way to insert logic into the class object creation process with the metaclass hook, other schemes are possible. They may not dovetail as neatly into the notion of metaclass methods we'll study ahead, but they do support creation-time tasks.

中文翻译

> 虽然重定义 type 超类的 new 与 init 是借助元类钩子向类对象创建过程插入逻辑的最常见方式，其他方案也可行。它们可能不如后文将学的元类方法概念契合，但确实支持创建时任务。

深度理解

·核心概念：metaclass= 只需可调用对象返回「像类的东西」；不必非是 type 子类。

·底层实现：函数、带 call 的普通实例都可充当。

·设计原因：协议本质是「callable(name, bases, dict) → class」。

·解决什么实际问题：快速原型、与装饰器边界模糊时的实验。

·初学者误区：生产代码滥用非 type 元类——会失去元类方法/继承树等能力。

40.15 使用简单工厂函数（Using simple factory functions）

英文原文

> For example, metaclasses need not really be classes at all. As we've learned, the class statement issues a simple call to create a class at the conclusion of its processing. Because of this, any callable object can, in principle, be used as a metaclass, provided it accepts the arguments passed and returns an object compatible with the intended class. In fact, a simple object factory function may serve just as well as a type subclass. When run, the function is called at the end of the declaring class statement, and it returns the expected ne

w class object. The function is simply catching the cal l that the type object's call normally intercepts by def ault. Technically speaking, such a plain function used as a metaclass can return anything: whatever it return s is assigned to the new class's name, whether it's a t ype instance or not. Other kinds of results may not s upport later instance creation, but this blurs the disti nction between metaclasses and class decorators—b oth rebind names in the end.

中文翻译

例如，元类其实根本不必是类。如我们所学，class 语句在处理结束时发出一次简单调用来创建类。因此原则上任何可调用对象都可用作元类，只要接受传入参数并返回与目标类兼容的对象。事实上，简单的对象工厂函数可以与 type 子类一样好用。运行时，函数在声明 class 语句末尾被调用，并返回期望的新类对象。该函数只是接住了通常由 type 的 call 默认拦截的那次调用。技术上，用作元类的普通函数可返回任何东西：返回值被赋给新类名，无论是否为 type 实例。其他结果可能不支持后续实例创建，但这模糊了元类与类装饰器的界限——二者最终都重绑定名字。

深度理解

·核心概念：函数元类 = 手动版 type.call 的替代接球手。

·底层实现：def MetaFunc(classname, supers, classdic t): return type(...)。

·设计原因：展示协议最小面；同时揭示与装饰器「最后都

是重绑定」的同源。

·解决什么实际问题：一次性创建时逻辑，无需定义 type 子类。

·初学者误区：以为函数元类也有「元类方法继承」——没有，因为没有次级 type 树角色。

代码分析

```
# Example 40-3. metaclass3.py
def MetaFunc(classname, supers, classdict):
    print('In MetaFunc:', classname, supers, classdict, sep='\n...…
    return type(classname, supers, classdict)

class Super:
    pass

print('Making class')
class Hack(Super, metaclass=MetaFunc):  #
    data = 1                            #
    def meth(self, arg):
        return self.data + arg

print('Making instance')
X = Hack()
print('Attrs:', X.data, X.meth(2))
# Making class → In MetaFunc: ... → Making instance → Attrs: 1 3
```

40.16 用普通类重载类创建调用（Overloading class creation calls with normal classes）

英文原文

Because normal (a.k.a. nonclass) instances can respon d to call operations with operator overloading, they c an serve in some metaclass roles, too, much like the preceding function. The output of Example 40-4 is si milar to the prior class-based versions, but it's based on a simple class—one that doesn't inherit from type at all and provides a call for its instances that catches the metaclass call using normal operator overloading . All classes are created from a metaclass, even if it's t he default type metaclass, but here we're using a non class instance of a class for the "metaclass." Note tha t new and init must use different names here, or else they will run when the MetaObj instance is created, n ot when that instance is later called in the role of met aclass after Hack's class statement. The call here mim ics part of what type's method does. When run, the t hree methods are dispatched via the normal instance 's call inherited from its normal class, but without any dependence on type dispatch mechanics or semantic s. This routing is largely about class alone. In fact, we can use normal superclass inheritance to acquire the call interceptor in this coding model—the superclass in Example 40-5 serves essentially the same role as ty pe, at least in terms of metaclass dispatch. Such code may be atypical, but it demos the underlying metacla ss dispatch model. Although such alternative forms w

ork, most metaclasses achieve their creation-time goals by redefining the type superclass's new and init; in practice, this may be simpler than other schemes. Regardless of its coding, though, this metaclass role broadly intersects with class decorators—as the next section will demonstrate.

中文翻译

因为普通（即非类）实例可用运算符重载响应调用操作，它们也可在某些元类角色中服务，很像前面的函数。示例 40-4 的输出与先前基于类的版本类似，但基于一个简单类——完全不继承 type，并为其实例提供 call，用普通运算符重载接住元类调用。所有类都由元类创建，即便是默认 type 元类；但这里我们用某个类的非类实例充当「元类」。注意此处 new/init 必须换名，否则会在创建 MetaObj 实例时运行，而非在 Hack 的 class 语句之后该实例作为元类被调用时。这里的 call 模仿了 type 方法的部分工作。运行时，三个方法经由普通实例从普通类继承的 call 分派，但不依赖 type 的分派机制或语义。这种路由大体只关于「类」。事实上，在此编码模型中可用普通超类继承获取调用拦截器——示例 40-5 中的超类在元类分派意义上本质上扮演与 type 相同的角色。此类代码可能非典型，但演示了底层元类分派模型。虽替代形式可行，多数元类通过重定义 type 的 new/init 达成创建时目标；实践中这可能比其他方案更简单。不过无论怎么写，这一元类角色都与类装饰器广泛相交——下一节演示。

深度理解

·核心概念：metaclass=MetaObj() 传入的是实例；实例的 call 充当造类工厂。

·底层实现：必须把仿 new/init 改名（如 New），避免与实例构造冲突。

·设计原因：证明「协议=可调用」；并展示普通继承即可组装分派。

·解决什么实际问题：教学演示；生产中仍推荐 class Meta(type)。

·初学者误区：metaclass=MetaObj（类）vs MetaObj()（实例）——本模式需要后者。

代码分析

```
# Example 40-4. metaclass4.py
class MetaObj:
    def __call__(self, classname, supers, classdict):
        print('In MetaObj.call:', classname, supers, classdict, se…
        Class = self.__New__(classname, supers, classdict)
        self.__Init__(Class, classname, supers, classdict)
        return Class
    def __New__(self, classname, supers, classdict):   #  __ne…
        print('In MetaObj.new: ', classname, supers, classdict, se…
        return type(classname, supers, classdict)
    def __Init__(self, Class, classname, supers, classdict):
        print('In MetaObj.init:', classname, supers, classdict, se…
        print('...init class object:', list(Class.__dict__.keys()))

class Hack(Super, metaclass=MetaObj()):  #
    data = 1
    def meth(self, arg):
```

```
        return self.data + arg

# Example 40-5 __call__
class SuperMetaObj:
    def __call__(self, classname, supers, classdict):
        Class = self.__New__(classname, supers, classdict)
        self.__Init__(Class, classname, supers, classdict)
        return Class

class SubMetaObj(SuperMetaObj):
    def __New__(self, classname, supers, classdict):
        return type(classname, supers, classdict)
    def __Init__(self, Class, classname, supers, classdict):
        ...
class Hack(Super, metaclass=SubMetaObj()):
    ...
```

40.17 用元类与装饰器管理类（Managing Classes with Metaclasses and Decorators)

英文原文

> Now that we understand the hook metaclasses use to insert code to be run at class construction time, let's put it to better use. In general, such code can be used to augment classes arbitrarily, and in many of the same ways as the class decorators of Chapter 39. This doesn't make these two tools identical—metaclasses also support the notion of inherited methods coming up later—but they are functionally redundant in some roles.

中文翻译

> 既然已理解元类用于在类构造时插入代码的钩子，让我们更好地使用它。一般而言，此类代码可任意增强类，且在许多方面与第 39 章的类装饰器相同。这并不使两工具等同——元类还支持后文的继承方法概念——但在某些角色上功能冗余。

深度理解

·核心概念：进入「实用对照」段：加方法、批量装饰方法。

·底层实现：双方都在 class 收尾拿到类（或 classdict）并改之。

·设计原因：用同一目标证明可互换，再引出元类独有能力。

·解决什么实际问题：工具库作者的选型实验场。

·初学者误区：以为「能互换」等于「永远该用装饰器」——后文有 twist。

40.18 向类添加方法（Adding methods to classes）

英文原文

To demo both this equivalence and more realistic usage, Example 40-6 uses metaclasses to augment the set of methods available in classes by inserting new m

ethods at class-creation time as sketched in the abstr act earlier—not the sort of thing most programmers do on a day-to-day basis, but potentially useful in to ols and libraries nonetheless. Recall again that class methods are simply functions that normally receive a n instance through which they are called. The metacl ass in this code leverages this to insert two functions into each of its client classes when those classes are made. The net effect provides methods and behavior for clients that do not exist in their class statements. Of course, the methods inserted here might be code d in a superclass and inherited by clients as usual, bu t the metaclass here is free to select inserted method s based on conditions tested whenever clients are bui lt. As covered ahead, we can also almost do the same with metaclass methods, but these methods are inher ited only by classes and wouldn't work in this exampl e; the methods inserted here are instead available to later class instances like X and Y. And while this may seem novel, it's easy to accomplish the same result w ith class decorators. As we've learned, there are indee d some striking similarities between these tools: Clas s decorators work by rebinding class names to the re sult of a callable at the end of a class statement after the new class has been created. Metaclasses work by routing class-object creation through a callable at th e end of a class statement in order to create the new class. The sum makes these two tools functionally eq

uivalent, at least in terms of class-creation dispatch. When run, this class-decorator version's output is identical to that of the metaclass variant in Example 40-6. It works the same because both decorator and metaclass are called at the end of a class statement and return an object to which the class's name is assigned. Decorators may be closest to the metaclass call because they are called to return an object, but like metaclass init, the class has already been built by the time a decorator runs. Metaclass call runs new and init, and metaclasses can augment in any of these three methods.

中文翻译

为演示这种等价性与更现实的用法，示例 40-6 用元类在类创建时插入新方法，以增强类可用方法集——如前抽象草图所述。多数程序员日常不做这种事，但对工具与库仍可能有用。再次回忆：类方法只是通常通过被调用的实例接收实例的函数。本代码中的元类利用这一点，在客户端类被制造时向每个客户端插入两个函数。净效果是为客户端提供其 class 语句中不存在的方法与行为。当然，插入的方法也可写在超类中由客户端照常继承，但此处元类可在客户端构建时按条件测试自由选择插入哪些方法。后文将述，我们几乎也可用元类方法做同样的事，但那些方法只被类继承，在本例中行不通；此处插入的方法则对后续类实例 X、Y 可用。虽看似新奇，用类装饰器很容易达到同样结果。如我们所学，两工具确有显著相似：类装饰器在 class 语句末尾、新类已创建后，把类名重绑定到可调用对象的结果；元类在 class

语句末尾把类对象创建路由通过可调用对象以创建新类。总和使两工具在类创建分派上至少功能等价。运行时，类装饰器版输出与示例 40-6 元类版相同。因为装饰器与元类都在 class 末尾被调用，并返回赋给类名的对象。装饰器可能最接近元类 call（被调用以返回对象），但像元类 init 一样，装饰器运行时类已建成。元类 call 运行 new 与 init，可在这三处任一增强。

深度理解

·核心概念：把函数塞进 classdict 或事后赋给类属性 → 实例可调用的普通方法。

·底层实现：元类在 new 改 dict；装饰器在类建成后 aClass.triple = triple。

·设计原因：实例需要的方法必须在主继承树（类 dict），不能只放在元类上。

·解决什么实际问题：工具向客户端注入 API 方法。

·初学者误区：用「元类方法」代替「插入实例方法」——可见性完全不同。

代码分析

```
# Example 40-6. extend_meta.py
def triple(obj):                 #
    return obj.value * 3
def concat(obj):                 #  obj  self
    return obj.value + 'Code!'

class Extender(type):
    def __new__(meta, classname, supers, classdict):
        classdict['triple'] = triple
```

```
        classdict['concat'] = concat
        return type.__new__(meta, classname, supers, classdict)

class Client1(metaclass=Extender):
    def __init__(self, value):
        self.value = value
    def double(self):
        return self.value * 2

class Client2(metaclass=Extender):
    value = 'grok'

X = Client1('hack')
print(X.double(), X.triple(), X.concat(), sep='\n')
# hackhack / hackhackhack / hackCode!
Y = Client2()
print(Y.triple(), Y.concat(), sep='\n')
# grokgrokgrok / grokCode!

# Example 40-7. extend_deco.py —
def extender(aClass):
    aClass.triple = triple
    aClass.concat = concat
    return aClass

@extender
class Client1:
    def __init__(self, value):
        self.value = value
    def double(self):
        return self.value * 2
```

40.19 自动装饰类方法（Automatically decorating class methods)

英文原文

Perhaps more interesting, metaclasses and decorators can both augment individual methods of classes. To demo, we'll use the utility module in Example 40-8, which resurrects the tracer and timer function decorators we coded in the prior chapter. This module is entirely review, so we'll defer to Chapter 39 for more details (and promise that this is the last mileage we'll get from this code). As we learned in Chapter 39, to use these decorators manually, we simply import them from the module and decorate each function we wish to augment. While this suffices for one-off augmentations, it requires us to add decoration syntax before each method we wish to trace or time and to later remove that syntax when we no longer desire the extensions. If we want to trace or time every method of a class, this can become tedious—and may not be possible at all in more dynamic contexts that depend upon runtime parameters. To do better, Example 40-9 uses a metaclass to add a decorator to each of a class's methods automatically. Because the metaclass controls decoration, it can predicate decoration on runtime checks. As a bonus, it can be used for any decoration: the decorator to apply to methods is passed as a top-level argument, and hence is allowed to vary per class. When this code is run as is, its output traces calls to every method of the client class because every met

hod has been automatically decorated by the metacl ass. Really, this result reflects a combination of decor ator and metaclass—the metaclass automatically app lies the function decorator to every method at class c reation time, and the function decorator automaticall y intercepts method calls in order to print the trace messages in this output. The combo "just works," tha nks to the generality of both tools. To apply a differe nt decorator to the methods, simply replace the deco rator name in the class header line. To use the timer f unction decorator shown earlier, for example, use eit her of the last two header lines in the following for o ur Person class—the first accepts the timer's default arguments, and the second specifies label text. And a s you might expect by now, class decorators intersect with metaclasses here, too. Example 40-10 replaces t he preceding example's metaclass with a class decora tor. Really, it uses a class decorator that applies a fun ction decorator to each method of a decorated class. Python's decorators naturally support arbitrary nestin g and combinations. When this code is run, the class decorator applies the tracer function decorator to ev ery method, which in turn produces a trace message on calls (the output is the same as that of the precedi ng metaclass version of this example). Notice that thi s class decorator returns the augmented class, not a proxy wrapper. As for the metaclass version, this retai ns the type of the original class—an instance of Pers

on is still an instance of Person. This may matter when type testing is used. The class's methods are not their original functions because they are rebound to decorators, but this is likely less important in practice, and it's true in the metaclass alternative as well. So far, what we've seen of metaclasses makes them seem largely redundant with decorators—but we have not yet seen all there is to see. As teased earlier, metaclasses may also provide behavior to their instance classes by defining methods, which have no direct counterpart in decorators. These methods, however, come with a twist that limits their scope. To understand both metaclasses' methods and their limiting twist, though, we first have to factor metaclasses into Python attribute resolution, a.k.a. inheritance, at large. The next section takes us down this prerequisite path.

中文翻译

或许更有趣的是，元类与装饰器都可增强类的各个方法。演示时，我们使用示例 40-8 的工具模块，它复活了上章编写的 tracer 与 timer 函数装饰器。该模块纯属复习，细节请回第 39 章（并保证这是这段代码的最后一次「里程」）。如第 39 章所学，手动使用这些装饰器只需导入并为每个欲增强的函数加装饰。这对一次性增强足够，但要求在每个欲追踪/计时的方法前加语法，并在不再需要时去掉。若想追踪或计时类的每个方法，会变得乏味——在依赖运行时参数的更动态上下文中甚至可能做不到。为做得更好，示例 40-9 用元类自动为类的每个方法添加装饰器。因元类控制装

饰，可按运行时检查决定是否装饰。额外好处是可用于任意装饰：施加到方法上的装饰器作为顶层参数传入，故可每类不同。按原样运行时，输出会追踪客户端类每个方法的调用，因为每个方法都被元类自动装饰了。实际上，这是装饰器与元类的组合——元类在类创建时自动把函数装饰器应用到每个方法，函数装饰器自动拦截方法调用以打印追踪消息。组合「就是能用」，归功于两工具的通用性。要对方法应用不同装饰器，只需替换 class 头中的装饰器名。例如要用先前的 timer，可用下列最后两行头之一——第一个接受 timer 默认参数，第二个指定标签文本。如你此刻可能预期的，类装饰器在此也与元类相交。示例 40-10 用类装饰器替换前例元类。实际上，它使用一个「把函数装饰器应用到被装饰类每个方法」的类装饰器。Python 装饰器天然支持任意嵌套与组合。运行时，类装饰器把 tracer 应用到每个方法，进而在调用时产生追踪消息（输出与前例元类版相同）。注意：该装装饰器返回增强后的类，而非代理包装。与元类版一样，这保留了原类类型——Person 的实例仍是 Person 的实例。在使用类型测试时这可能重要。类的方法不再是原始函数（已重绑定到装饰器），但实践中这通常不太重要，元类替代方案亦然。至此，我们所见使元类看似与装饰器大体冗余——但尚未看全。如前预告，元类还可通过定义方法为其实例类提供行为，装饰器无直接对等物。不过这些方法带有限制其作用域的 twist。要理解元类方法及其限制性 twist，我们必须先把元类纳入 Python 属性解析（即继承）的大局。下一节带我们走这条前置路径。

深度理解

·核心概念：decorateAll(decorator) 工厂返回元类或类

装饰器，遍历类命名空间，对 FunctionType 包装。

·底层实现：元类改 classdict 后 type.new；装饰器用 set attr（不要只改 dict 视图假设）。

·设计原因：一处开关切换 tracer/timer；运行时可谓词化。

·解决什么实际问题：整类埋点、性能探针、统一鉴权包装。

·初学者误区：用「带 call 的类」做方法装饰器会踩第 39 章方法陷阱；此处沿用函数装饰器。

代码分析

```
# Example 40-8. decorators.py
import time
def tracer(func):
    calls = 0
    def onCall(*args, **kwargs):
        nonlocal calls
        calls += 1
        print(f'call {calls} to {func.__name__}')
        return func(*args, **kwargs)
    return onCall

def timer(label='', trace=True):
    def onDecorator(func):
        def onCall(*args, **kargs):
            start = time.perf_counter()
            result = func(*args, **kargs)
            elapsed = time.perf_counter() - start
            onCall.alltime += elapsed
            if trace:
                print(f'{label}{func.__name__}: {elapsed:.5f}, {on…
            return result
```

```
        onCall.alltime = 0
        return onCall
    return onDecorator

# Example 40-9. decoall_meta.py
from types import FunctionType
from decorators import tracer, timer

def decorateAll(decorator):
    class MetaDecorate(type):
        def __new__(meta, classname, supers, classdict):
            for attr, attrval in classdict.items():
                if type(attrval) is FunctionType:
                    classdict[attr] = decorator(attrval)
            return type.__new__(meta, classname, supers, classdict)
    return MetaDecorate

class Person(metaclass=decorateAll(tracer)):
    def __init__(self, name, pay):
        self.name = name
        self.pay = pay
    def giveRaise(self, percent):
        self.pay *= (1.0 + percent)
    def lastName(self):
        return self.name.split()[-1]

#
# class Person(metaclass=decorateAll(timer())):
# class Person(metaclass=decorateAll(timer(label='**'))):

# Example 40-10. decoall_deco.py
def decorateAll(decorator):
    def DecoDecorate(aClass):
        for attr, attrval in aClass.__dict__.items():
            if type(attrval) is FunctionType:
                setattr(aClass, attr, decorator(attrval))  #  …
        return aClass
```

```
    return DecoDecorate

@decorateAll(tracer)
class Person:
    ...
```

40.20 继承：终章（Inheritance: The Finale）

英文原文

Because metaclasses are coded in similar ways to inheritance superclasses, their scope can be confusing at first glance. In short, there are really two class trees searched by Python inheritance—a primary tree formed by a class and that class's superclasses, along with a secondary tree formed by a class's metaclass and that metaclass's superclasses. The secondary tree is also called the "type" tree because it stems from type. In more detail, here is how this pans out: Metaclasses inherit from the type class (usually). Although they have a special role, metaclasses are coded with normal class statements and follow the usual OOP model in Python. As subclasses of type, they can redefine the type object's methods to customizing classes as needed. Per the prior section, metaclasses often redefine the type class's new and init to intercept class creation and initialization. Metaclasses can also define methods for their instance classes and may be simple func

tions or other callables that return arbitrary objects. Metaclass attributes are not acquired by class instanc es. Metaclass declarations specify an instance relatio nship, which is not quite the same as superclass inher itance. Behavior defined in a metaclass applies to the classes made from it but not to these classes' own no nclass instances. Inheritance for a nonclass instance s earches only the instance and the primary tree forme d by its class and that class's superclasses; the secon dary metaclass tree is never included in this search. H ence, nonclass instances get behavior from classes an d superclasses but not from metaclasses. Metaclass a ttributes are acquired by classes as a fallback. By cont rast, classes do acquire methods of their metaclasses by virtue of the instance relationship. Metaclasses de fine a separate inheritance tree, which is a source of c lass behavior that processes classes themselves. For c lasses only, inheritance first searches the primary tree formed by the class and its superclasses and then fall s back on the secondary tree formed by the class's m etaclass and its superclasses as a separate search. Wh en a name is available to a class in both a metaclass a nd a superclass, the superclass version is used. Metac lass declarations are also inherited by subclasses. The metaclass=M declaration in a user-defined class is als o inherited by the class's normal subclasses in much t he same way that superclass links are inherited by su bclasses. Thus, the metaclass will run for the construc

tion of each class that inherits this specification in a s
uperclass inheritance chain. This model's conflation o
f classes and metaclasses can make terminology chall
enging, but it may be easier to understand in code th
an in prose. When this file's code is run (as a script or
module), the metaclass handles construction of both
client classes. When those classes are later used, non
class instances inherit class attributes but not metacl
ass attributes. By contrast, classes both inherit names
from their superclasses and acquire names from their
metaclass—whose linkage in this example is itself inh
erited from a superclass by Sub. Notice how the last
of the preceding calls fails when we pass in an instan
ce because the name resolves to a metaclass method,
not a normal instance method. In fact, both the sourc
e of a name and the object through which you fetch i
t matter here. Methods acquired from metaclasses ar
e bound to the subject class, while methods from nor
mal classes are plain functions when fetched through
the class but bound with an instance when fetched th
rough the instance.

中文翻译

因为元类的编码方式与继承超类相似，其作用域乍看可能令人困惑。简言之，Python 继承实际搜索两棵类树——由类及其超类形成的主树，以及由类的元类及其超类形成的次树。次树也称为「type 树」，因为它源于 type。更细地说：元类通常继承 type 类。虽有特殊角色，元类用普通 class

语句编写并遵循 Python 常规 OOP 模型。作为 type 子类，它们可重定义 type 的方法以按需定制类。按前节，元类常重定义 new/init 以拦截类创建与初始化。元类也可为其实例类定义方法，并可是返回任意对象的简单函数或其他可调用对象。元类属性不被类的实例获取。元类声明指定的是实例关系，与超类继承并不完全相同。元类中定义的行为适用于由其制造的类，但不适用于这些类自己的非类实例。非类实例的继承只搜索实例及其类与超类形成的主树；次级元类树从不包含在此搜索中。因此非类实例从类和超类获得行为，不从元类获得。元类属性由类作为回退获取。相比之下，类凭借实例关系确实获取其元类的方法。元类定义独立继承树，是处理类自身的类行为来源。仅对类而言，继承先搜索类及其超类的主树，再回退到类的元类及其超类的次树作为单独搜索。当一名在元类与超类中都可用时，使用超类版本。元类声明也被子类继承。用户定义类中的 metaclass=M 声明也会被该类的普通子类继承，方式与超类链接被子类继承大体相同。因此，元类会为超类继承链中继承该规格的每个类的构造而运行。该模型对类与元类的混同可能使术语具有挑战性，但用代码可能比用散文更好理解。当该文件代码运行（脚本或模块）时，元类处理两个客户端类的构造。这些类随后使用时，非类实例继承类属性但不继承元类属性。相比之下，类既从超类继承名字，也从元类获取名字——本例中该链接本身由 Sub 从超类继承。注意：当前面最后一次调用传入实例时会失败，因为名字解析为元类方法，而非普通实例方法。事实上，名字来源与你通过哪个对象获取它都重要。从元类获取的方法绑定到主体类；从普通类获取的方法在通过类获取时是普通函数，通过实例获取时与实例绑定。

深度理解

·核心概念：双树模型——主树（实例可见）+ 次树（仅类可见）；metaclass= 沿子类传递。

·底层实现：X.meth3 失败；Sub.meth3() 成功且为 bound method on class。

·设计原因：隔离「类工具 API」与「实例 API」，避免元类方法污染业务实例。

·解决什么实际问题：解释抽象基类、ORM 模型类上的类级 API 从何而来。

·初学者误区：以为「类有的实例都有」——对元类名字不成立。

代码分析

```
# Example 40-11. metainstance.py
class Meta(type):
    def __new__(meta, classname, supers, classdict):
        print('In Meta.new:', classname)
        return type.__new__(meta, classname, supers, classdict)
    def meth3(self):
        return 'three!'

class Super(metaclass=Meta):   #
    def meth2(self):
        return 'two!'

class Sub(Super):              #  vs
    def meth1(self):
        return 'one!'

# import  Meta SuperSub
```

```
>>> X = Sub()
>>> X.meth1(), X.meth2()       #
('one!', 'two!')
>>> X.meth3()                  #
AttributeError: ...

>>> Sub.meth1(X), Sub.meth2(X), Sub.meth3()
('one!', 'two!', 'three!')     #
>>> Sub.meth3(X)               #
TypeError: Meta.meth3() takes 1 positional argument but 2 were giv…

>>> Sub.meth3   # bound to class
<bound method Meta.meth3 of <class 'metainstance.Sub'>>
>>> Sub.meth2   #
<function Super.meth2 at ...>
>>> X.meth2     #
<bound method Super.meth2 of <...Sub object...>>
```

40.21 元类 vs 超类（Metaclass Versus Superclass）

英文原文

In even simpler terms, watch what happens in the following: as an instance of the M metaclass type, class C acquires M's attribute, but this attribute is not made available for inheritance by C's own instance I—the acquisition of names by metaclass instances is distinct from the normal inheritance used for nonclass instances. By contrast, if M morphs from metaclass to superclass, then names in superclass S become available to later instances of C by inheritance—that is, by sear

ching the dict attribute dictionaries of objects in the MRO of I's class as usual. This is why metaclasses oft en do their work by manipulating a new class's name space dictionary if they wish to influence the behavio r of later instance objects—instances will see names i n their class but not its metaclass. Watch what happe ns, though, if the same name is available in both attri bute sources—the inheritance name in the primary s uperclass tree is used instead of the instance acquisit ion name in the secondary metaclass tree. This is true regardless of the relative height of the inheritance an d instance sources—superclass inheritance always be ats metaclass acquisition because primary trees are s earched before secondary trees. In fact, classes acqui re metaclass attributes through their class link, in the same way that nonclass instances inherit from classes through their class—which makes sense, given that cl asses are also instances of metaclasses. The chief dist inction is that instance inheritance does not also follo w a class's class but instead restricts its scope to the dict of each class in the superclass tree, following bas es along the way. Really, though, Python checks the dict of each class on the class's MRO before falling b ack on doing the same for its metaclass—and runs th e second of these steps only for fetches run on classe s, not nonclass instances. These two MROs for class a nd metaclass are simply the flattened versions of wha t we called the primary and secondary class trees earl

ier. Nonclass-instance inheritance searches just the first, but class inheritance searches both if needed. In fact, metaclasses work in much the same way, as the next section explains.

中文翻译

用更简单的话说，观察以下情况：作为 M 元类类型的实例，类 C 获取 M 的属性，但该属性不会提供给 C 自己的实例 I 继承——元类实例获取名字，与非类实例使用的普通继承是不同的。相比之下，若 M 从元类变形为超类，则超类 S 中的名字可通过继承供 C 的后续实例使用——即照常搜索 I 的类的 MRO 中各对象的 dict。这就是为何元类若希望影响后续实例对象的行为，常通过操纵新类的命名空间字典开展工作——实例会看到其类中的名字，但看不到其元类中的名字。不过若同一名字在两个属性源都可用——主超类树中的继承名会优先于次元类树中的实例获取名。无论继承源与实例源的相对高度如何都成立——超类继承总是击败元类获取，因为先搜索主树再搜索次树。事实上，类通过其 class 链接获取元类属性，方式与非类实例通过 class 从类继承相同——这说得通，因为类也是元类的实例。主要区别是：实例继承不会再跟随类的 class，而是把范围限制在超类树中每个类的 dict，沿 bases 前进。实际上，Python 在回退到对其元类做同样事情之前，先检查类的 MRO 上每个类的 dict——且第二步仅对在类上运行的获取执行，不对非类实例。类与元类的这两条 MRO 只是我们先前所称主、次类树的扁平化版本。非类实例继承只搜索第一条；类继承在需要时搜索两条。事实上元类工作方式大体相同，下一节解释。

深度理解

·核心概念：同名冲突时超类永远赢元类；实例永不走类的 class。

·底层实现：C.attr 可来自 M；I.attr 仅当 attr 在 C/S 的主 MRO 中。

·设计原因：保证业务继承语义稳定，元类只是类级工具层。

·解决什么实际问题：选型——要影响实例就改 classdict/插方法；只要类 API 就放元类。

·初学者误区：在元类上放 attr 期望 I.attr 可用。

代码分析

```
>>> class M(type): attr = 1
>>> class C(metaclass=M): pass
>>> I = C()
>>> C.attr          # 1 —
>>> I.attr          # AttributeError —
>>> 'attr' in C.__dict__, 'attr' in M.__dict__
(False, True)

#  M  S
>>> class S: attr = 1
>>> class C(S): pass
>>> I = C()
>>> C.attr, I.attr  # (1, 1) —

#
>>> class M(type): attr = 1
>>> class S: attr = 2
>>> class C(S, metaclass=M): pass
```

```
>>> C.attr, I.attr  # (2, 2)

#
>>> class S2: attr = 2
>>> class S1(S2): pass
>>> class C(S1, metaclass=M): pass
>>> C.attr, C().attr  # (2, 2)

>>> C.__class__     # <class 'M'> —  instance-of
>>> C.__bases__     #  —
>>> [x.__name__ for x in C.__mro__]   #  MRO
>>> [x.__name__ for x in M.__mro__]   #  MRO
```

40.22 元类继承（Metaclass Inheritance）

英文原文

As it turns out, instance inheritance works in similar ways, whether the "instance" is a nonclass instance cr eated from a class or a class created from a metaclas s derived from type. While classes straddle the prima ry and secondary trees, both trees use the same mec hanism to look up names. This yields a single attribut e search procedure spanning two trees, which fosters the parallel notion of metaclass inheritance hierarchi es. The following demos this conceptual merger. In it, instance I inherits from all its classes; class C inherits from both superclasses and metaclasses; and metacla ss M1 inherits from higher metaclasses. The failure at the end of this listing is pivotal. Both inheritance pat hs—class and metaclass—employ the same links to b

uild class MROs scanned by inheritance. But this is not applied transitively—instances do not inherit their class's metaclass names, though they may request them explicitly. And as usual, both trees have flattened MROs and instance links actually used by inheritance. If you care about metaclasses—or must use code that does—study these examples carefully. In effect, nonclass instances have no bases to search but follow class to bases once, and classes follow bases before following a single class to another bases. Because this is crucial to the meaning of attribute names in Python, the next section begins making it more formal.

中文翻译

事实证明，无论「实例」是由类创建的非类实例，还是由派生自 type 的元类创建的类，实例继承工作方式相似。类横跨主、次两树，但两树用同一机制查找名字。这产生跨越两树的单一属性搜索过程，并促成元类继承层级的平行概念。下列演示这一概念合并。其中，实例 I 从其所有类继承；类 C 从超类与元类继承；元类 M1 从更高元类继承。列表末尾的失败是关键的。两条继承路径——类与元类——使用相同链接构建由继承扫描的类 MRO。但这不传递应用——实例不继承其类的元类名字，虽可显式请求它们。一如既往，两树都有扁平化 MRO 以及继承实际使用的实例链接。若你关心元类——或必须使用涉及元类的代码——请仔细研究这些例子。实际上，非类实例没有 bases 可搜索，但跟随 class 到 bases 一次；类先跟随 bases，再跟随单一 class 到另一 bases。因这对 Python 中属性名的含义至关重

要，下一节开始更形式化。

深度理解

·核心概念：元类也可有自己的继承层级（M1←M2）；类合并两树；实例不传递跟随。

·底层实现：I.class.attr4 显式路由到元类树；I.attr4 失败。

·设计原因：单一算法、两棵树、一次「跨树」仅允许在类上。

·解决什么实际问题：读懂复杂框架中多层元类。

·初学者误区：以为 I 自动拥有 C 从元类获得的一切。

代码分析

```
>>> class M2(type): attr4 = 4      #
>>> class M1(M2): attr3 = 3
>>> class S: attr2 = 2             #
>>> class C(S, metaclass=M1): attr1 = 1
>>> I = C()
>>> I.attr1, I.attr2               # (1, 2)
>>> C.attr1, C.attr2, C.attr3, C.attr4  # (1,2,3,4)
>>> M1.attr3, M1.attr4             # (3, 4)
>>> I.attr3                        # AttributeError
>>> I.__class__.attr4              # 4 —
>>> [x.__name__ for x in C.__mro__]   # ['C','S','object']
>>> [x.__name__ for x in M1.__mro__]  # ['M1','M2','type','object']
```

40.23 Python 继承算法：简单版（Python Inheritance Algorithm: The Simple Version

)

英文原文

Now that we know about metaclass acquisition, we're finally able to formalize the inheritance rules that the y augment. Inheritance deploys two distinct but simil ar lookup routines, both of which are founded on MR Os computed from the prior section's bases links per Chapter 32. The combination yields the following firs t-cut definition of Python's attribute inheritance algo rithm run for explicit attribute fetches. To look up an attribute name: 1. From a nonclass instance I, search the instance, then its class, and then all its superclass es, using: (a) The dict of the instance I; (b) The dict of all classes on the mro found at I's class, from left to ri ght. 2. From a class C, search the class, then all its su perclasses, and then its metaclasses tree, using: (a) T he dict of all classes on the mro found at C itself, fro m left to right; (b) The dict of all metaclasses on the mro found at C's class, from left to right. 3. In rules 1 and 2, also allow for these exceptions covered ahead: Give precedence to data descriptors present in step b sources; Skip step a and begin the search at step b fo r built-in operations; Perform a custom MRO search f or a proxied object in super objects. Rules 1 and 2 ar e applied for normal, explicit attribute fetch only, and there are exceptions for built-ins, descriptors, and su per, which we'll clarify in a moment. In addition, a get

attr or getattribute may also be used for missing or a ll names, respectively, per Chapter 38, and attribute a ssignment follows different rules. The first two rules here, however, are the essentials. They specify the inh eritance search of two separate trees, which works th e same in each tree but spans trees for class inherita nce alone. The MRO pseudocode of Chapter 32 sum marizes this even more concisely: [I.dict] + [x.dict for x in I.class.mro] for instances; [x.dict for x in C.mro] + [x.dict for x in C.class.mro] for classes. In this, the first line is sources searched by rule 1 for inheritance from a nonclass instance I—both the instance itself and th e flattened version of the primary class tree. The seco nd line is rule 2's sources for inheritance from a class C—the flattened versions of both the primary class tr ee and the secondary metaclass tree. Put another wa y, nonclass instances and classes both search a class t ree's MRO at their class as a second step, but classes first search their own MRO's superclass tree instead o f a single dict namespace dictionary. In code, class he ader lines define both of the trees searched by listing superclasses and specify links to secondary trees with metaclass keywords. When omitted, superclass defau lts to object and metaclass to type. Most programme rs need only be aware of the first of these rules (1) an d perhaps the first step of the second (2a). There's an extra acquisition step added for metaclasses (2b), but it's essentially the same as others—a subtle equivale

nce, to be sure, but metaclass acquisition is not as novel as it may seem. It's just one step of the procedure.

中文翻译

既然已了解元类获取，我们终于能形式化它们所增强的继承规则。继承部署两套不同但相似的查找例程，都建立在按第 32 章由先前节的 bases 链接计算的 MRO 上。组合给出 Python 对显式属性获取运行的属性继承算法的初稿定义。查找属性名：1. 从非类实例 I：搜索实例、其类、及其所有超类，使用：(a) 实例 I 的 dict；(b) 在 I 的 class 找到的 mro 上所有类的 dict，从左到右。2. 从类 C：搜索类、其所有超类、及其元类树，使用：(a) 在 C 自身找到的 mro 上所有类的 dict，从左到右；(b) 在 C 的 class 找到的 mro 上所有元类的 dict，从左到右。3. 在规则 1 与 2 中，还允许后文所述例外：给步骤 b 源中出现的数据描述符优先；对内置运算跳过步骤 a 从 b 开始；对 super 对象中的代理对象执行定制 MRO 搜索。规则 1 与 2 仅适用于正常、显式属性获取；内置、描述符与 super 有例外，稍后澄清。此外，按第 38 章，getattr 或 getattribute 可分别用于缺失名或所有名；属性赋值遵循不同规则。不过此处前两条规则是本质。它们指定对两棵独立树的继承搜索，每棵树内工作方式相同，但仅类继承跨越树。第 32 章的 MRO 伪代码更简洁地总结：实例为 [I.dict] + [x.dict for x in I.class.mro]；类为 [x.dict for x in C.mro] + [x.dict for x in C.class.mro]。第一行是规则 1 从非类实例 I 继承搜索的源——实例自身与主类树的扁平化版本。第二行是规则 2 从类 C 继承的源——主类树与次元类树的扁平化版本。换言之，

非类实例与类都在第二步搜索其 class 处类树的 MRO，但类首先搜索自身 MRO 的超类树，而非单一 dict 命名空间字典。代码中，class 头通过列出超类定义被搜索的树，并用 metaclass 关键字指定次树链接。省略时，超类默认为 object，元类默认为 type。多数程序员只需知道规则 1，或许还有规则 2 的第一步 (2a)。为元类增加了额外获取步骤 (2b)，但本质上与其他步骤相同——确实是微妙的等价，但元类获取并不如看起来那么新奇。它只是过程中的一步。

深度理解

·核心概念：全书继承算法初稿——实例单树；类双树；例外另列。

·底层实现：伪代码两行即心智模型；默认 object / type 补全两端。

·设计原因：先给工程师可用的主路径，再在「Less Simple」中叠加描述符等。

·解决什么实际问题：调试 AttributeError、预测 C.x vs I.x。

·初学者误区：把 2b 当成日常实例查找的一步。

40.24 描述符偏误（The descriptors deviation）

英文原文

At least, that's the normal—and sugarcoated—case. T
he prior section listed its exceptions separately becau
se they don't matter in most code and complicate th
e algorithm substantially. First among these, inherita
nce also has a special-case coupling with Chapter 38'
s attribute descriptors. In short, data descriptors—th
ose that define set methods to intercept assignments
—are given precedence, such that their names overri
de other inheritance sources. This exception ostensibl
y serves some practical roles. For example, it is used t
o ensure that the special class and dict attributes can
not be redefined by the same names in an instance's
own dict. In the following, these data-descriptor nam
es in the class override same names created in the in
stance's attribute dictionary. This data-descriptor exc
eption is tested before the preceding two inheritance
rules as a preliminary step, may be more important t
o Python implementers than Python programmers, a
nd can be reasonably ignored by most application co
de in any event—that is, unless you code data descri
ptors of your own, which follow the same inheritance
special case. Conversely, if this descriptor did not defi
ne a set, the name in the instance's dictionary would
hide the name in its class instead, per normal inherita
nce. In both cases, Python automatically runs the des
criptor's get when it's found by inheritance rather tha

n returning the descriptor object itself—part of the implicit attribute magic we met earlier in the book. The special status afforded to data descriptors, however, also modifies the meaning of attribute inheritance and, thus, the meaning of names in your code. The next section's final swing at a formal inheritance algorithm takes this into account.

中文翻译

至少，那是正常——且被糖衣包裹——的情形。前节单独列出例外，因为它们在多数代码中无关紧要却大幅复杂化算法。其中首先，继承还与第 38 章的属性描述符有特例耦合。简言之，数据描述符——定义 set 以拦截赋值的那些——被赋予优先权，使其名字覆盖其他继承源。该例外表面上服务一些实际角色。例如，用于确保特殊的 class 与 dict 属性不能由实例自身 dict 中的同名项重定义。下列中，类中的这些数据描述符名覆盖在实例属性字典中创建的同名。该数据描述符例外作为预备步骤在前两条继承规则之前测试，对 Python 实现者可能比对程序员更重要，多数应用代码无论如何可合理忽略——除非你自己编写数据描述符，它们遵循同一继承特例。相反，若该描述符未定义 set，则实例字典中的名字会按普通继承隐藏类中的名字。两种情况下，Python 在由继承找到描述符时自动运行其 get，而非返回描述符对象本身——这是本书早先遇到的隐式属性魔法的一部分。但赋予数据描述符的特殊地位也修改了属性继承的含义，从而修改了代码中名字的含义。下一节对正式继承算法的最后一击会将此纳入。

深度理解

·核心概念：数据描述符（有 set）优先于实例 dict；非数据描述符可被实例项遮挡。

·底层实现：即使 I.dict['class']='hack'，I.class 仍走类上数据描述符。

·设计原因：保护对象模型关键槽位；property/slots 等建立在此上。

·解决什么实际问题：理解为何「往实例 dict 塞同名」甩不掉某些属性。

·初学者误区：以为实例字典永远最优先——对数据描述符不成立。

代码分析

```
>>> class C: pass
>>> I = C()
>>> I.__dict__['book'] = 'lp6e'       #
>>> I.__dict__['__class__'] = 'hack'  #
>>> I.__dict__['__dict__'] = {}
>>> I.book                            # 'lp6e'  dict
>>> I.__class__, I.__dict__           #  dict

>>> class D:
...     def __get__(self, instance, owner): print('D.__get__')
...     def __set__(self, instance, value): print('D.__set__')
>>> class C: d = D()                  #
>>> I = C(); I.d                      # D.__get__
>>> I.__dict__['d'] = 'hack'
>>> I.d                               #  D.__get__ –

>>> class D:                          #
```

```
...     def __get__(self, instance, owner): print('D.__get__')
>>> class C: d = D()
>>> I = C(); I.__dict__['d'] = 'hack'
>>> I.d                                  # 'hack' –
```

40.25 Python 继承算法：不那么简单版（Python Inheritance Algorithm: The Less Simple Version）

英文原文

With both the data descriptor special case and general descriptor invocation factored in with class and metaclass trees, Python's formal inheritance algorithm can be stated as follows—a complex procedure which assumes knowledge of descriptors, metaclasses, and MROs but is the final arbiter of attribute names nonetheless. To look up an attribute name: 1. From a nonclass instance I, search the instance, its class, and its superclasses, as follows: (a) Search the dict of all classes on the mro found at I's class; (b) If a data descriptor was found in step a, call its get and exit; (c) Else, return a value in the dict of the instance I; (d) Else, call a nondata descriptor or return a value found in step a. 2. From a class C, search the class, its superclasses, and its metaclasses tree as follows: (a) Search the dict of all metaclasses on the mro found at C's class; (b) If a data descriptor was found in step a, call its get and

exit; (c) Else, call any descriptor or return a value in th e dict of a class on C's own mro; (d) Else, call a nonda ta descriptor or return a value found in step a. 3. In r ules 1 and 2, built-in operations essentially use just st ep a sources (see ahead). 4. The super built-in perfor ms a custom MRO search for a proxied object (see ah ead). Some fine print here: in this procedure, items ar e attempted in sequence as numbered, and Python r uns at most one (for instances) or two (for classes) M RO searches per name lookup—the first appearance of a name in a given MRO wins, regardless of its kind . Because each MRO (a.k.a. mro) is a flattened represe ntation of a class tree with duplicates removed, you c an also think of these as one or two tree searches. In addition, the implied object superclass provides som e defaults at the top of every class and metaclass tre e (that is, at the end of every MRO). And beyond all t his, method getattr may be run if defined when an at tribute is not found, and method getattribute may be run for every attribute fetch, though they are special- case extensions to the name lookup model (really, th e latter replaces inheritance for the defining class's in stances, and can trigger the former with an attribute exception). See Chapter 38 for more on these tools a s well as descriptors. Also, note here again that this a lgorithm's first two steps apply only to normal and ex plicit attribute fetch. The rules for attribute assignme nt vary; the implicit lookup of method names for buil

t-ins doesn't follow these rules in full; and the proxied lookup of attributes performed by super is entirely custom. The next sections cover these exceptions separately.

中文翻译

在把数据描述符特例与一般描述符调用连同类与元类树一并纳入后，Python 的正式继承算法可陈述如下——这是一个复杂过程，假定你了解描述符、元类与 MRO，但它仍是属性名的最终仲裁者。查找属性名：1. 从非类实例 I：(a) 搜索 I 的 class 上 mro 所有类的 dict；(b) 若在 a 找到数据描述符，调用其 get 并退出；(c) 否则返回实例 I 的 dict 中的值；(d) 否则调用非数据描述符或返回 a 中找到的值。2. 从类 C：(a) 搜索 C 的 class 上 mro 所有元类的 dict；(b) 若 a 找到数据描述符，调用 get 并退出；(c) 否则调用任一描述符或返回 C 自身 mro 上某类 dict 中的值；(d) 否则调用非数据描述符或返回 a 中的值。3. 规则 1 与 2 中，内置运算本质上只用步骤 a 的源（见后）。4. 内建 super 对代理对象执行定制 MRO 搜索（见后）。细则：过程中按编号顺序尝试各项；每次名字查找 Python 最多运行一次（实例）或两次（类）MRO 搜索——给定 MRO 中名字首次出现即胜，不论其种类。因每个 MRO 是去重后的类树扁平表示，也可把这些想成一或两次树搜索。此外，隐含的 object 超类在每个类与元类树顶端（即每个 MRO 末尾）提供一些默认。在此之外，若定义了 getattr，属性未找到时可运行；getattribute 可对每次属性获取运行——它们是名字查找模型的特例扩展（后者实际上替换定义类之实例的继承，并可以属性异常触发前者）。描述符等详见第 38 章。再次

注意：算法前两步仅适用于正常显式属性获取。赋值规则不同；内置的方法名隐式查找不完全遵循这些规则；super 的代理查找完全定制。下一节分别涵盖这些例外。

深度理解

·核心概念：完整算法把数据描述符优先织进双树搜索；类查找甚至先扫元类树找数据描述符。

·底层实现：实例路径「先类 MRO 找数据描述符 → 再实例 dict → 再类 MRO 普通值」；类路径对称地先元类。

·设计原因：实现层保护与 property 语义；代价是心智负担。

·解决什么实际问题：解释 slots/property 与实例赋值的交互。

·初学者误区：死记步骤编号而不做实验——应用代码多只需 simple version。

40.26 赋值附录（The assignment addendum）

英文原文

The prior section defines inheritance in terms of attribute reference (a.k.a. fetch or lookup), but parts of it apply to attribute assignment as well. As we've learned, assignment normally changes attributes in the subject object itself, but inheritance is also invoked on as

signment to test first for some of Chapter 38's attribute management tools, including descriptors, properties, and setattr. When present, such tools intercept attribute assignment and may implement it arbitrarily. For example, attribute assignment always invokes a setattr if present, much as a getattribute is run for all references. Moreover, a data descriptor with a set method is acquired from a class by inheritance using the MRO and has precedence over the normal storage model. In terms of the prior section's rules: When applied to an instance, attribute assignments essentially follow steps a through c of rule 1, searching the instance's superclass tree, though step b calls set instead of get, and step c stops and stores in the instance instead of attempting a fetch. When applied to a class, attribute assignments run the same procedure on the class's metaclass tree: roughly the same as rule 2, but step c stops and stores in the class. Because descriptors are also the basis for other advanced attribute tools such as properties and slots, this inheritance precheck on assignment is utilized in multiple contexts. The net effect is that descriptors are treated as an inheritance special case for both reference and assignment.

中文翻译

前节以属性引用（获取/查找）定义继承，但部分也适用于属性赋值。如我们所学，赋值通常改变主体对象自身的属性

，但赋值时也会调用继承，以首先测试第 38 章的一些属性管理工具，包括描述符、property 与 setattr。若存在，这些工具拦截赋值并可能任意实现它。例如，若存在 setattr，属性赋值总是调用它，正如 getattribute 对所有引用运行。此外，带 set 的数据描述符通过 MRO 从类继承获取，并优先于普通存储模型。按前节规则：应用于实例时，属性赋值本质上遵循规则 1 的步骤 a 到 c，搜索实例的超类树，但步骤 b 调用 set 而非 get，步骤 c 停止并存入实例而非尝试获取。应用于类时，属性赋值在类的元类树上运行相同过程：大致如规则 2，但步骤 c 停止并存入类。因为描述符也是 property 与 slots 等其他高级属性工具的基础，赋值上的这种继承预检在多种上下文中被利用。净效果是：描述符在引用与赋值上都视为继承特例。

深度理解

·核心概念：赋值不是「只写本地 dict」——先继承探测数据描述符/setattr。

·底层实现：实例赋值扫主 MRO；类赋值扫元类树相关路径。

·设计原因：让 obj.x = 1 能触发 property setter。

·解决什么实际问题：调试「赋值进了哪里」。

·初学者误区：以为 I.dict['x']=1 与 I.x=1 总等价——后者走描述符协议。

40.27 super 补充（The super supplement）

英文原文

> Even for attribute reference, there are two special cases that are exempt from inheritance's normal rules. For one, the super built-in function we studied in Chapter 32 does not use normal inheritance. As we learned, for the proxy objects returned by super, attributes are resolved by a special context-sensitive scan of a limited tail portion of a different object's MRO. This custom scan searches the MRO of an implicit instance's class, choosing the first descriptor or value found in a class following the class containing the super call. This scan is used instead of running full inheritance—which is used on the super object itself only if this special-case scan fails. See Chapter 32 for more coverage of super. While this built-in may be convenient in some simple roles, it comes with substantial complexity in others and adds a convoluting footnote to inheritance itself.

中文翻译

即便对属性引用，也有两个特例豁免于继承的正常规则。其一，我们在第 32 章学过的内建 super 函数不使用普通继承。如我们所学，对 super 返回的代理对象，属性通过对其另一对象 MRO 有限尾部的特殊上下文相关扫描来解析。该

定制扫描搜索隐式实例之类的 MRO，选择在包含 super 调用的类之后的类中找到的第一个描述符或值。用此扫描代替完整继承——仅当此特例扫描失败时，才对 super 对象本身使用完整继承。更多见第 32 章。虽该内建在某些简单角色中可能方便，在其他角色中带来可观复杂性，并给继承本身添加曲折脚注。

深度理解

·核心概念：super() 代理 ≠ 普通属性查找；走 MRO 尾部协作式调用。

·底层实现：从「当前类之后」继续，支持菱形多重继承协作。

·设计原因：合作方法（cooperative methods）需要跳过已参与的类。

·解决什么实际问题：super().init 链；也是继承算法必须单列的原因。

·初学者误区：把 super().f 想成「父类的 f」——多继承下是「MRO 下一家」。

40.28 内置分岔（The built-ins bifurcation）

英文原文

As we've also learned, other built-ins don't follow inheritance's normal rules either. Instances and classes may both be skipped for the implicit method-name fetches of built-in operations, as a special case that di

ffers from explicit name references. Because this is a context-specific divergence, it's easier to demonstrate in code than to weave it into a single algorithm. In the following, str is the built-in, str is its explicit-name equivalent, and the nonclass instance is inconsistently skipped by the built-in only. As you may expect by now, the same holds true for classes—explicit names start at the class, but built-ins start at the class's class—which is its metaclass, and defaults to type, which provides access to an implicit default. In fact, it can sometimes be nontrivial to know where a name comes from in this model since all classes in both trees also inherit from object—including the default type metaclass. In the following's explicit call, C gets a default str from object instead of the metaclass, per the first source of class inheritance (the class's own MRO, which is the primary tree); by contrast, the str built-in skips ahead to the metaclass as before. This is why we've gone to such great lengths to root out the full specs of inheritance. While some code may never need to care about all its many plot twists, attribute inheritance is clearly a convoluted business in Python, and uncertainty is not generally compatible with engineering.

中文翻译

如我们亦已学过，其他内置也不遵循继承的正常规则。实例与类都可能对内置运算的隐式方法名获取被跳过，这是有别于显式名字引用的特例。因这是上下文相关的分岔，用代码

演示比织入单一算法更容易。下列中，str 是内建，str 是其显式名等价物，非类实例仅被内建不一致地跳过。如你此刻可能预期，对类同样成立——显式名从类开始，但内建从类的类开始——即其元类，默认为 type，提供对隐式默认的访问。事实上，有时要知道此模型中名字从何而来并不平凡，因为两树中所有类也继承自 object——包括默认 type 元类。下列显式调用中，按类继承的第一源（类自身 MRO，即主树），C 从 object 获得默认 str 而非元类；相比之下，str 内建仍像以前一样跳到元类。这就是为何我们不遗余力挖出继承的完整规格。虽有些代码可能永远不必关心其诸多情节转折，属性继承在 Python 中显然是曲折业务，而不确定性通常与工程不相容。

深度理解

·核心概念：显式 I.str vs 内建 str(I) 分岔；对类则是主树 vs 元类。

·底层实现：内建方法查找常跳过实例 dict，直接从类型对象（类或元类）槽位取。

·设计原因：速度与一致性（类型上的 C 槽）；副作用是代理类难「完全透明」。

·解决什么实际问题：解释第 39 章代理为何要手工转发内建；元类上的 str 如何影响 str(C)。

·初学者误区：在实例上设 str 却期望 print(obj) 变化——对某些模式不成立；应在类上定义。

代码分析

```
>>> class C:
...     attr = 1
...     def __str__(self): return('class')
>>> I = C()
>>> I.__str__(), str(I)              # ('class', 'class')
>>> I.__str__ = lambda: 'instance'
>>> I.__str__(), str(I)              # ('instance', 'class')
>>> I.attr = 2; I.attr               #

#
>>> class D(type):
...     def __str__(self): return('D class')
>>> class C(D): pass
>>> C.__str__(C), str(C)             # ↛↛(type)
>>> class C(metaclass=D):
...     def __str__(self): return('C class')
>>> C.__str__(C), str(C)             # ('C class', 'D class') …

>>> class C(metaclass=D): pass
>>> C.__str__(C), str(C)             # ↛object↛ D
```

40.29 继承收束（The Inheritance Wrap-Up）

英文原文

And with all those details in hand, you finally have th e complete Python inheritance story—or at least as much as we can cover in this text. It's a tangled tale t hat today spans instances, classes, superclasses, meta classes, descriptors, super, built-ins, and MROs, and a

ll for the sake of looking up a simple attribute name. Some practical needs warrant exceptions, of course, but you should carefully consider the implications of an object-oriented language that applies inheritance—its foundational operation—in such a labyrinthian fashion. At a minimum, this should underscore the importance of keeping your own code simple to avoid making it dependent on the darker corners of such convoluted rules. As always, your code's users and maintainers will be glad you did. For more fidelity on this story, see Python's internal implementation of inheritance—a low-level but complete saga chronicled today in its files object.c and typeobject.c, the former for normal instances and the latter for classes. Delving into internals shouldn't be required to use Python, but it's the ultimate and sometimes sole source of truth in a complex and perpetually changing system. This is especially true in boundary cases born of accrued exceptions that raise the bar for learners and users, a downside we'll revisit briefly in the next and closing chapter. For now, let's move on to one last bit of metaclass "magic"—its methods, which rely on its inheritance offshoot.

中文翻译

掌握所有这些细节后，你终于拥有完整的 Python 继承故事——或至少是本文能覆盖的全部。这是一个纠缠的故事，今天横跨实例、类、超类、元类、描述符、super、内置与

MRO，而一切只为查找一个简单的属性名。当然，一些实际需求证明例外合理，但你应仔细考虑：一门面向对象语言以如此迷宫方式应用其基础操作——继承——意味着什么。至少，这应强调保持自有代码简单的重要性，以免依赖如此曲折规则的阴暗角落。一如既往，你的代码用户与维护者会因此感激。若需更高保真，请看 Python 继承的内部实现——低层但完整的传奇，如今记载于 object.c（普通实例）与 typeobject.c（类）。深入内部本不应是使用 Python 的前提，但在复杂且持续变化的系统中，它是最终、有时是唯一的真相来源。对由累积例外产生的边界情形尤其如此——它们抬高了学习者与用户的门槛，这一 downside 我们将在下一章也是收官章中短暂重访。现在，让我们进入元类「魔法」的最后一点——其方法，它们依赖其继承分支。

深度理解

·核心概念：完整继承 = 双树 + 描述符 + 赋值预检 + super 定制 + 内置分岔。

·底层实现：真相在 CPython 的 object.c / typeobject.c。

·设计原因：历史演进叠加特例；工程上应用「保持简单」对抗复杂度。

·解决什么实际问题：给工具作者一张地图；给应用作者一条「别踩角落」的忠告。

·初学者误区：追求在业务代码中炫技触发所有特例。

40.30 元类方法（Metaclass Methods）

英文原文

Now that we have a handle on the way that metaclasses modify the inheritance of names, we can finally turn to their methods with full clarity. In short, methods in metaclasses are inherited by and process their instance classes—instead of the nonclass instances that classes make. This makes metaclass methods similar in form and function to the class methods we studied in Chapter 32, though their class-focused behavior is automatic. Moreover, they are available only to classes due to the last section's inheritance rules—the limiting "twist" alluded to earlier. Metaclass methods also have no direct analog in class decorators, though decorators' freedom to return arbitrary objects makes nearly anything possible with imagination. To demo the basics, the following's metaclass defines a method made available to its class's instances by metaclass acquisition (i.e., by inheritance from the secondary metaclass tree). Methods fetched from the class are plain functions as usual, but those from a metaclass are automatically bound to the class from which they were fetched, as we saw in an earlier example. As we've also seen, methods fetched through a nonclass instance are bound to the instance unless classmethod or staticmethod are used, though such instances are moo

t here because they do not inherit metaclass names. The only real new things about metaclass methods is that they are inherited only by classes and are bound to a class automatically. The latter half of this is explored in the next section.

中文翻译

既然已掌握元类如何修改名字继承，我们终于能完全清晰地转向其方法。简言之，元类中的方法由其实例类继承并处理实例类——而非类所制造的非类实例。这使元类方法在形式与功能上类似第 32 章的类方法，但其面向类的行为是自动的。此外，因上节继承规则，它们仅对类可用——即先前提及的限制性「twist」。元类方法在类装饰器中也无直接类比，不过装饰器可返回任意对象，凭想象几乎什么都行。演示基础：下列元类定义一个方法，通过元类获取（即从次元类树继承）提供给其类的「实例」（即类）。从类获取的方法一如既往是普通函数，但从元类获取的方法自动绑定到获取它们的类，如我们在先前例子中所见。如我们亦见，通过非类实例获取的方法绑定到实例（除非使用 classmethod/staticmethod），不过此处此类实例无关紧要，因为它们不继承元类名字。关于元类方法真正新鲜的只有：它们仅被类继承，并自动绑定到类。后一半在下一节探索。

深度理解

·核心概念：元类方法 = 自动的、仅类可见的「隐式 classmethod」。

·底层实现：C.z → bound method of class；I.z → AttributeError。

·设计原因：为类对象提供工具 API 而不污染实例命名空间。

·解决什么实际问题：类注册、类级校验、类上运算符（见后）。

·初学者误区：给 I 传参调用元类方法。

代码分析

```
>>> class M(type):
...     def z(cls): print('M.z', cls)   # =
...     def y(cls): print('M.y', cls)   #  C
>>> class C(metaclass=M):
...     def y(self): print('C.y', self)
...     def x(self): print('C.x', self)
>>> C.x, C.y                            # function
>>> C.z                                 # bound method on C
>>> C.z()                               # M.z <class C>
>>> I = C()
>>> I.x(); I.y()                        #
>>> I.z()                               # AttributeError
```

40.31 元类方法 vs 类方法（Metaclass Methods Versus Class Methods）

英文原文

> Though they differ in inheritance visibility, metaclass methods are designed to manage class-level data, just like the class methods we studied in Chapter 32. In fact, their roles can overlap—much as metaclasses can in general with class decorators—but metaclass meth

ods are not accessible except through the class and do not require an explicit classmethod class-level declaration in order to be bound with the class. In other words, metaclass methods can be thought of as implicit class methods, with limited visibility. This yields two very different ways to create class methods whose asymmetry is left as suggested pondering here. NOTE: Plus one more: Python 3.6 added an operator-overloading method named initsubclass. This method is called whenever the containing class is subclassed, and it receives a single argument: the new subclass object. It provides yet another way to manage classes, though this method is nowhere near as general as class decorators or metaclasses: it's run only for a class's own subclasses and only when those subclasses are created. Similar to metaclass methods, this method is also implicitly converted to a class method. Because we just can't get enough of those implicit hooks!

中文翻译

虽继承可见性不同，元类方法旨在管理类级数据，正如第 32 章的类方法。事实上其角色可重叠——正如元类总体上可与类装饰器重叠——但元类方法除通过类外不可访问，且不需要显式 classmethod 声明即可与类绑定。换言之，元类方法可视为隐式类方法，且可见性受限。这产生两种非常不同的创建类方法的方式，其不对称性留作此处建议的思考。注意：还有一个：Python 3.6 增加了名为 initsubclass 的运算符重载方法。每当包含它的类被 subclass 时调用，

接收单一参数：新子类对象。它提供又一种管理类的方式，但远不如类装饰器或元类通用：仅对类自己的子类运行，且仅在那些子类创建时。与元类方法类似，该方法也隐式转换为类方法。因为我们就是对那些隐式钩子欲罢不能！

深度理解

·核心概念：@classmethod 实例可见；元类方法仅类可见；initsubclass 是第三钩子。

·底层实现：C.a() 写类数据；I.b() 可调 classmethod；I.a() 失败。

·设计原因：多种钩子服务不同可见性/时机需求。

·解决什么实际问题：选 API 暴露面——给实例用 classmethod，仅给类用元类方法。

·初学者误区：三者混用却不测可见性。

代码分析

```
>>> class M(type):
...     def a(cls):                 #
...         cls.x = cls.y + cls.z
>>> class C(metaclass=M):
...     y, z = 11, 22
...     @classmethod
...     def b(cls):                 #
...         return cls.x
>>> C.a()                           #
>>> C.x                             # 33 —
>>> I = C()
>>> I.x, I.y, I.z                   # (33, 11, 22)
>>> I.b()                           # 33 — classmethod
>>> I.a()                           # AttributeError —
```

40.32 元类方法中的运算符重载（Operator Overloading in Metaclass Methods）

英文原文

> Just in case your head is not yet spinning, metaclasses, just like normal classes, may also employ operator overloading to make built-in operations applicable to their instance classes. The getitem indexing method in the following metaclass, for example, is a metaclass method designed to process classes themselves—the classes that are instances of the metaclass, not those classes' own nonclass instances. And if that's not abstruse enough, when both class and metaclass overload the same operator, the former applies to classes and the latter to nonclass instances—wait: actually per the built-ins bifurcation, built-ins on the class object look to the metaclass, while built-ins on instances look to the class. All of which leads us to the closing compare-and-contrast of the next section.

中文翻译

以防你的头还没晕：元类与普通类一样，也可运用运算符重载，使内置运算适用于其实例类。例如下列元类中的 getitem 索引方法，是设计来处理类自身的元类方法——即作为元类实例的那些类，而非那些类自己的非类实例。若这还不够深奥：当类与元类重载同一运算符时，结合内置分岔—

—对类对象的内建看向元类，对实例的内建看向类。这一切把我们引向下一节的收官对照。

深度理解

·核心概念：C[i] 可走元类 getitem；I[i] 走类上定义（若有）。

·底层实现：内建分岔 + 元类方法可见性共同作用。

·设计原因：让「类也像对象」支持 C[0] 这类 DSL。

·解决什么实际问题：枚举类、配置类的下标访问等。

·初学者误区：期望 I[0] 自动获得元类索引。

代码分析

```
>>> class M(type):
...     def __getitem__(cls, i):    #
...         return cls.data[i]
>>> class C(metaclass=M):
...     data = 'hack'
>>> C[0]                            # 'h'
>>> C.__getitem__                   # bound on class
>>> I = C()
>>> I.data, C.data                  #
>>> I[0]                            # TypeError: not subscriptable

>>> class C(metaclass=M):
...     data = 'hack'
...     def __getitem__(self, i):
...         return self.data[i]
>>> I = C(); I.data = 'code'
>>> C[0], I[0]                      # ('h', 'c') — [][]
```

40.33 元类方法 vs 实例方法（Metaclass Methods Versus Instance Methods）

英文原文

The inescapable conclusion of this technical novel is that metaclass methods provide a separate—and arguably redundant—way to code object behavior with classes in Python. As we've learned, the usual way to implement objects is with classes and their nonclass instances. Here's the normal case—with class data, constructor, regular method, and operator overloading available to instances and subclasses. As usual, we make an instance of a normal class like this by calling it with constructor arguments. And to make more instances, we simply call the class again. The equivalent metaclass can also provide data, regular methods, and operator overloading. But per-instance constructors don't quite apply; methods receive and process a class, not instances of it; and this behavior is for subclasses and instance classes in the secondary "type" tree only. Because metaclass instances are classes, we define a new class per instance instead of running a call and code instance data as class attributes, though all the behavior defined in the metaclass applies to its instance classes. We haven't made any normal instances here—just a class that nevertheless serves as an information record with both data and behavior metho

ds. Coding differences aside, the main functional dive rgence in the metaclass approach is that nonclass ins tances created from such a class don't inherit any of t he behavior defined in a metaclass. To make addition al instances in the metaclass world, we must instead code additional classes. Calling the metaclass with no arguments doesn't work at all because it makes a cla ss, not an instance of a class—though calling the met aclass with full type arguments does work because it' s equivalent to running a class statement (and yes, th e fact that metaclasses are classes, too, makes some of this paragraph's logic circular, but that's just how Python works: metaclasses are special-cased for meta class roles). In sum, metaclasses allow us to impleme nt classes that work much like the nonclass instances we've used throughout this book. As to why you'd w ant to swap one kind of instance for another with ide ntical functionality but different coding, we'll have to defer to other resources to justify the redundancy. Gi ven the many convolutions that metaclasses bring to the table, it's difficult not to see this as complexity fo r all in the name of rare and narrow roles. While that has been a recurring theme in both Python and this b ook, it's time to wrap up and move on to the conclusi on.

中文翻译

这部技术小说不可避免的结论是：元类方法提供了用类编码对象行为的一条分离——且arguably 冗余——的途径。如我们所学，实现对象的通常方式是类及其非类实例。这是正常情形——类数据、构造器、常规方法与运算符重载对实例与子类可用。一如既往，通过带构造参数调用普通类来制造实例。要更多实例，只需再次调用类。等价的元类也可提供数据、常规方法与运算符重载。但「每实例构造器」并不完全适用；方法接收并处理的是类，而非其实例；且该行为仅用于次「type」树中的子类与实例类。因为元类实例是类，我们为每个「实例」定义一个新类，而不是运行一次调用，并把「实例数据」编码为类属性；不过元类中定义的全部行为都适用于其实例类。我们这里没有制造任何普通实例——只是一个仍作为带数据与行为方法的信息记录的类。抛开编码差异，元类途径的主要功能分歧是：由此类创建的非类实例不继承元类中定义的任何行为。要在元类世界制造更多「实例」，我们必须改而编码更多类。无参调用元类完全不行，因为它制造类而非类的实例——不过用完整 type 参数调用元类可行，因为等价于运行 class 语句（是的，元类也是类这一事实使本段部分逻辑循环，但这就是 Python 的工作方式：元类为元类角色被特殊对待）。总之，元类允许我们实现工作方式很像本书通篇使用的非类实例的类。至于为何要用编码不同、功能相同的另一种实例去替换，我们只能把为这种冗余辩护留给其他资源。鉴于元类带来的诸多曲折，很难不把这看成以稀有而狭窄的角色为名、加诸所有人的复杂性。虽这是 Python 与本书反复出现的主题，但现在是收束并走向结论的时候了。

深度理解

·核心概念：同一领域模型可用「普通实例」或「类当记录 + 元类当行为」表达；后者对普通实例不可见。

·底层实现：class pat(metaclass=MetaEmployee) 代替 pat = Employee(...)；再「实例」就要再写一个 class 或 MetaEmployee(name, bases, dict)。

·设计原因：Lutz 的批评立场——为稀有角色付出全局复杂度。

·解决什么实际问题：理解为何多数项目停在装饰器/class method，不上元类 DSL。

·初学者误区：用元类重写本可用普通类表达的业务对象。

代码分析

```
#  OOP
class Employee:
    rate = 50
    def __init__(self, name, hours):
        self.name = name
        self.hours = hours
    def pay(self):
        print(f'{self.name} ${self.rate * self.hours:,.2f}')
    def __iadd__(self, hours):
        self.hours += hours
        return self

pat = Employee('Pat', 2_000)
pat.pay()          # Pat $100,000.00
pat += 1_000
pat.pay()          # Pat $150,000.00
pat2 = Employee('Pat2', 1_000)
```

```
# ——=
class MetaEmployee(type):
    rate = 50
    def pay(cls):
        print(f'{cls.name} ${cls.rate * cls.hours:,.2f}')
    def __iadd__(cls, hours):
        cls.hours += hours
        return cls

class pat(metaclass=MetaEmployee):
    name = 'Pat'
    hours = 2_000
pat.pay(); pat += 1_000; pat.pay()

pat2 = pat()       #
pat2.pay()         # AttributeError —

class pat2(metaclass=MetaEmployee):  #
    name = 'Pat2'
    hours = 1_000
# pat2 = MetaEmployee('pat2', (), dict(name='Pat2', hours=1_000))
```

40.34 本章小结（Chapter Summary）

英文原文

In this chapter, we studied metaclasses, explored exa mples of them in action, and formalized the rules of i nheritance that they extend. Along the way, we also s aw how the roles of class decorators and metaclasses often intersect: because both run at the conclusion of a class statement, they can sometimes be used interc

hangeably. We also learned how metaclass methods define behavior for classes much like the class methods we met earlier, but are limited in scope to the secondary inheritance tree searched for classes. Since this chapter covered an advanced topic, we'll work through just a few quiz questions to review the basics (candidly, if you've made it this far in a chapter on metaclasses, you probably already deserve extra credit!). Because this is the last part of the book, we'll also forgo the end-of-part exercises. Be sure to see the appendixes that follow for Python platform pointers and the solutions to the prior parts' exercises; the last of these includes a sampling of short application-level programs for self-study. Once you finish the quiz, you've officially reached the end of this book's technical material. The next and final chapter offers some brief thoughts about Python to wrap up the book at large and closes with some fun. We'll regroup there in the Learning Python benediction after you work through this final quiz.

中文翻译

在本章中，我们研究了元类，探索了它们的运行示例，并形式化了它们所扩展的继承规则。一路上，我们也看到类装饰器与元类的角色如何经常相交：因为二者都在 class 语句收尾运行，有时可以互换使用。我们还学到，元类方法为类定义行为，很像先前遇到的类方法，但作用域限于为类搜索的次级继承树。因本章覆盖高级主题，我们只做几个测验题复

习基础（坦率说，若你已读到元类章的这里，大概已该得额外学分！）。因这是书的最后一部分，我们也略去部分末练习。请务必查看随后的附录，以获取 Python 平台指引与先前各部分练习的解答；其中最后一部分包含若干供自学的短小应用级程序。完成测验后，你就正式到达本书技术材料的终点。下一章也是最后一章将提供关于 Python 的简短思考以收束全书，并以一些趣味收尾。在你做完这最后测验后，我们将在《Learning Python》的祝福中再会合。

深度理解

·核心概念：三线收束——元类钩子、与装饰器重叠、完整继承 + 元类方法 twist。

·底层实现：class 收尾双通道（Meta 调用 vs 装饰器重绑定）。

·设计原因：工具作者地图；应用作者「保持简单」。

·解决什么实际问题：衔接结论章与附录。

·初学者误区：以为没写过元类就没学完 Python——理解模型比会写框架更重要。

40.35 测验（Test Your Knowledge: Quiz）

英文原文

1. What is a metaclass? 2. How do you declare the metaclass of a class? 3. How do class decorators overlap with metaclasses for managing classes?

中文翻译

> 1. 什么是元类？2. 如何声明一个类的元类？3. 在管理类方面，类装饰器如何与元类重叠？

深度理解

·核心概念：三问覆盖定义、语法、与装饰器对照——本章最小可交付理解。

·底层实现：答案见下一节。

·设计原因：高级章用短测验收束，避免再加项目级练习。

·解决什么实际问题：自检是否抓住主线。

·初学者误区：只背术语不写 metaclass= 小实验。

40.36 测验答案（Test Your Knowledge: Answers）

英文原文

1. A metaclass is a class used to create a class. Classes are instances of the type class by default. Metaclasses are usually subclasses of the type class, which customize classes. They may redefine class-creation protocol methods like new and init to customize the class creation call issued at the end of a class statement. They may also define data and methods that provide behavior for their instance classes, but these are inherited only by classes, not their nonclass instances. Met

aclasses can also be coded in other ways—as simple functions, for example—but they are responsible for making and returning an object for the new class. Finally, metaclasses constitute a secondary pathway for inheritance search used for classes alone; this allows classes to ape normal instance behavior, though it bifurcates the class story for relatively rare and narrow roles. 2. Use a keyword argument in the class header line: class C(metaclass=M). The class header line can also name normal superclasses before the metaclass keyword argument; these superclasses are searched before metaclasses for inheritance run from a class. 3. Because both are automatically triggered at the end of a class statement, class decorators and metaclasses can both be used to manage classes. Decorators rebind a class name to a callable's result, and metaclasses route class creation through a callable, but both hooks can be used for similar purposes. To manage classes, decorators simply augment and return the original class objects. Metaclasses augment a class after or before they create it. As noted, metaclasses can also provide behavior for their instance classes in the form of methods that are not immediately supported by decorators, though the objects returned by decorators can do anything supported by the Python language.

中文翻译

> 1. 元类是用来创建类的类。类默认是 type 类的实例。元类通常是 type 的子类，用于定制类。它们可重定义 new、init 等类创建协议方法，以定制 class 语句末尾发出的类创建调用。它们也可定义数据与方法，为其实例类提供行为，但这些仅被类继承，不被其非类实例继承。元类也可用其他方式编码——例如简单函数——但它们负责制造并返回新类的对象。最后，元类构成仅用于类的次级继承搜索路径；这允许类模仿普通实例行为，却也为相对稀有而狭窄的角色分岔了类的故事。2. 在 class 头行使用关键字参数：class C(metaclass=M)。头行也可在 metaclass 关键字参数之前列出普通超类；从类运行继承时，这些超类在元类之前被搜索。3. 因为二者都在 class 语句末尾自动触发，类装饰器与元类都可用于管理类。装饰器把类名重绑定到可调用对象的结果，元类把类创建路由通过可调用对象，但两种钩子可用于相似目的。为管理类，装饰器只需增强并返回原类对象。元类在创建类之前或之后增强类。如前所述，元类还能以方法形式为其实例类提供行为，装饰器并不直接支持（不过装饰器返回的对象可做 Python 语言支持的任何事）。

深度理解

·核心概念：答案四句口号——造类的类；metaclass=；收尾双钩子；元类方法仅类可见。

·底层实现：与 40.6–40.10、40.18–40.20、40.30 呼应。

·设计原因：把整章压缩成可口述的三答。

·解决什么实际问题：面试/自检清单。

·初学者误区：第 3 题只答「一样」——需点出元类方法

这一差异。

本章总结

技术拓展（Technical Expansion）

·实际项目中的应用场景

· ORM / 模型声明：Django models、SQLAlchemy 声明式基类用元类（或等价钩子）收集字段属性、生成映射表。

· API 框架：校验器批量挂到方法、路由注册、插件类自动发现（initsubclass 常作轻量替代）。

·测试与可观测性：decorateAll(tracer|timer) 模式——整类埋点而不改每个方法。

·枚举与数据类演进史：Enum 元类约束、早期 ORM 与现代 dataclass/initsubclass 的消长。

·与其他语言的区别

语言	近似机制	与 Python 元类差异
Python	type 子类 / metaclass=	运行时造类，类也是对象
Java	注解处理器、字节码增强	多在编译期；无统一「类的类」一等公民模型
C++	模板元编程	编译期类型计算，无运行时 class 对象协议
Ruby	开放类 + 单例类	同样「类是对象」，钩子形态不同
Smalltalk	显式元类	历史渊源；Python 的 type 统一更「单根」

· Python 发展历史背景：新式类统一后 type 成为默认元类；metaclass= 关键字在 Python 3 取代旧式 __metaclass__ 赋值；3.6 起 __initsubclass__ 与 __setname__ 为许多「只需子类钩子」场景减负，使「不必动元类」的工具更多。

·高级开发者相关知识

主题	作用
完整属性查找（本章算法）	读框架、写描述符/代理不踩内置分岔
类装饰器 vs 元类选型	实例管理→装饰器；创建协议/元类方法→元类
types.newclass / typing	动态类与静态类型的现代接口
CPython typeobject.c	边界行为的最终出处
initsubclass	多数「注册子类」场景的首选轻量钩子

学习建议（Learning Advice）

·重要程度：4/5 星（工具作者必读；应用作者重在理解）。作为全书最后技术章，它收官继承，并把装饰器对照补全；日常业务少写元类，但读 Django/SQLAlchemy/枚举/ABC 必须懂模型。

·应该掌握到什么程度

1. 能画出「实例 → 类 → type」与「主 MRO + 次 MRO」两张图。

2. 能手写最小 class Meta(type) 的 __new__/__init__，并说明何时用哪个。

3. 能把「加方法 / 批量装饰方法」分别用元类与类装饰器各写一版。

4. 能解释为何 C.attr 可见而 I.attr 不可见（元类属性），以及超类为何总优先于元类。

5. 能口述 simple inheritance 两行伪代码，并知道描述符、赋值、super、内置四个例外的存在。

6. 能对比元类方法、classmethod、initsubclass 的可见性与时机。

·后续学习方向

1. 结论章：把「复杂性与特例累积」放回 Python 全局图景。

2. 重读第 32（类是对象）、第 38（描述符）、第 39（装饰器）与本章对照。

3. 阅读 CPython 或精简教学实现中的属性查找伪代码；对一个自写描述符跑遍 I.x/C.x/str 实验。

4. 在真实框架里 grep metaclass= / initsubclass，用本章地图标注每处钩子角色。

5. 优先用类装饰器与 initsubclass 解决需求；仅当需要创建协议或类级次树行为时再上元类。

> 全章完。元类是「类的类」，继承是「两棵树加一串例外」。把 type(name, bases, dict)、metaclass=、双树查找与装饰器对照亲手跑通，你就真正读完了《Learning Python》的技术主线。